Primula author guide

Primula Documenting System 0.4.31 · edition of 11.11.2026

Contents

Primula author guide....6
Pages....6
What you need....6
How a document is made....7
1. First document....8
1.1. File....8
1.2. YAML header....8
1.3. Text and paragraphs....8
1.4. Headings, clauses, lists....9
1.5. Check and build....9
1.6. Highlighting in other editors....9
1.7. Default look....10
2. Contract....11
2.1. The whole document....11
2.2. Data go into the header, not into the text....12
2.3. Beginning of the contract....12
2.4. Sections and clauses....12
2.5. References to clauses....12
2.6. Sums and numbers....13
2.7. Requisites and signatures....13
2.8. Appendix....13
2.9. Footnotes....13
2.10. Repeated blocks....13
3. Act and tables with totals....14
3.1. A simple table....14
3.2. Act: rows from data and a total....14
3.3. A table without a loop....15
3.4. A cell of several paragraphs....15
3.5. Look of a table....15
4. Official letter....17
4.1. Example....17
4.2. Parts of the letter....17
4.3. Your own differences....18
5. Technical documentation....19
5.1. Admonitions and notes....19
5.2. Variants for different versions....19
5.3. Abbreviations....20
5.4. Index, table of contents, lists....20
5.5. Figures....20

5.6. Program text and interface....21
5.7. Shared fragments and keys....21
6. Project of several documents....22
6.1. Project file....22
6.2. A document of the project....22
6.3. Fragments: shared text....23
6.4. Modules: macros and functions....23
6.5. Data from files....24
6.6. Building the project....24
6.7. Export to Markdown, HTML, DOCX....24
6.8. Quick view: primula view....25
6.9. Assembled document....25
6.10. References to another document....25
6.11. Key map....26
6.12. Order project....26
7. Styles....27
7.1. Choosing a set....27
7.2. Applying styles in the text....27
7.3. Your own styles module....28
7.4. Layers....28
7.5. Pages, headers and footers....29
7.6. Moving from ODT Markdown Kit....29
8. Messages of the translator....30
8.1. How to read a message....30
8.2. Warnings and the release....30
8.3. Common errors....30
8.4. When it is not clear what is wrong....32
9. Parallel texts....33
9.1. Example....33
9.2. Tracks....33
9.3. Segments....34
9.4. Numbering, references, footnotes....34
9.5. Correspondence check....35
9.6. Translation: blanks, confirmation, freshness....35
9.7. Protocol of disagreements....36
9.8. Styles....36
10. Code: the Python dialect....37
10.1. Where code is written....37
10.2. Numbers: decimal instead of int and float....37
10.3. Strings....37
10.4. Lists, tuples, dictionaries....38

10.5. Records....38
10.6. Dates and lengths....39
10.7. Functions....39
10.8. Global names: lazy and immutable....40
10.9. Modules and fragments....40
10.10. Values in the text and macros....41
10.11. What the core does not have....42
11. Libraries: development and use....43
11.1. The libraries of the kit....43
11.2. What a library can do....44
11.3. Directory layout....44
11.4. Module....45
11.5. Tests....45
11.6. Using a library in a project....46
11.7. Data with a date of effect....46
11.8. What is inside a file: primula describe....47
11.9. Release....47
11.10. Commands of a module: |def, param, the accumulator....47
12. Importing documents from Word and other formats....48
12.1. The command and the Studio window....48
12.2. What is carried over....48
12.3. How the structure is recognized....49
12.4. Word styles....49
12.5. The result....50
12.6. What to do after the import....50
Appendix A. Constructs....51
Commands....51
Short forms....53
Closing markers....53
Appendix B. Functions of expressions....55
Numbers....55
Money and VAT....55
In words....56
Dates and time....56
Units and lengths....56
Text....57
Lists and records....57
Data....58
Mathematics (std/math)....58
QR codes and encodings....59
Checks and document....59

Table formulas....59
Styles code....60
Project....61
Methods....61
Appendix C. YAML header keys....63
Appendix D. Messages of the translator....66
Appendix E. Styles of the kit....77
std/styles/default....77
std/styles/business....79
std/styles/letter....80
std/styles/technical....81
std/styles/report....81
std/styles/compact....82
Глоссарий терминов Primula (ru / uk / en)....84

Primula author guide

A guide for those who write documents in Primula: contracts, acts, letters, instructions and reports. It explains how to solve a typical task and does not repeat the reference tables. The complete lists of constructs, functions, header keys, messages and styles are in the appendices, which are generated from the translator and always match its version.

The normative documents are the specification `primula-spec-v1.1-draft.ru.md` and the description of the standard library `primula-stdlib-v1.1-draft.ru.md`; their English translations are `primula-spec-v1.1-draft.en.md` and `primula-stdlib-v1.1-draft.en.md`. A comment at the top of every page of the guide names the specification sections it relies on.

Pages

Page	About
First document	a `.prml` file, the YAML header, paragraphs, headings, clauses, check and build
Contract	parameters, place and date, preamble, sections and clauses, references to clauses, sums in words, requisites, appendix
Act and tables with totals	tables, typed columns, formulas and totals, rows from data
Official letter	the `letter` styles set: letterhead, addressee, number and date, signature
Technical documentation	admonitions, conditional variants, abbreviations, index, figures
Project of several documents	`project.p.prml`, shared parameters, fragments, modules, build targets, orders
Styles	styles sets of the kit, your own styles module, applying styles, moving from Kit
Messages of the translator	how to read messages, common errors and what to do
Parallel texts	bilingual agreement, protocol of disagreements: tracks and segments, language and numbering of a track, correspondence check, confirmation of a translation
Code: the Python dialect	`decimal` numbers, strings, collections, records, dates, functions, lazy immutable global names, modules and export from a fragment (`@export`), values in the text, what the core does not have
Libraries: development and use	a library of modules and data, what it can and cannot use, the directory layout, test documents with `fail()`, linking by a symbolic link, a copy or a Git submodule (decision 136), data with a date of effect, release
Importing documents	`primula import`: DOCX, ODT, DOC, Markdown, HTML → a Primula draft; recognition of the structure, Word styles, `import-styles.yaml`, what to do after the import

Appendices (generated, not edited by hand): constructs, functions of expressions, header keys, messages, styles of the kit. Terms follow the glossary.

What you need

- The translator `primula` — from the package or from the source tree (`./primula`). PDF needs LibreOffice.
- An editor: Primula Studio (highlighting, hints, check, PDF preview) or any text editor with UTF-8.

Studio is described in its own help (F1 in Studio); the guide shows the translator commands that Studio runs for you.

How a document is made

A document is a plain text file. Text is written as it is, and everything that belongs to markup starts with a vertical bar `|`: `|<` — a heading, `|>` — a clause of a contract, `|(сума)` — a value from code, `|*{важливо}` — emphasis. The translator checks the file, computes the values, numbers the clauses and references and writes an ODT document (and a PDF through LibreOffice). The look is set by styles, not in the text: the same document prints differently when another styles set is used.

1. First document

1.1. File

A Primula document is a text file in UTF-8 with the extension `.prml`. Files of other kinds may lie next to it; the sub-extension tells them apart:

File	Kind
`договір.prml`	document — an ODT is built from it
`умови.f.prml`	fragment — a piece of text inserted with `|include`
`бізнес.m.prml`	module — macros and functions for `|import`
`стилі.s.prml`	styles module
`project.p.prml`	project file

1.2. YAML header

The file starts with a header between `---` lines. It holds the data of the document, without computations:

```
---
doc:
  title: "Довідка про співпрацю"
  language: uk-UA
params:
  організація: "ТОВ «Приклад»"
  рік: 2026
---
|(організація) співпрацює з нами з |(рік) року.
```

- `doc:` — ODT metadata: title, language, author, number, revision and so on. In the text they are available as `|(doc.title)`. The language `uk-UA` sets the quotes, hyphenation, the format of dates and of sums in words.
- `params:` — parameters of the document. The text and the code see them by name. A parameter can be replaced when building: `primula build довідка.prml --param рік=2027`.

All keys are listed in the appendix “Header keys”.

1.3. Text and paragraphs

Paragraphs are separated by a blank line. A line break inside a paragraph is a space, so a long paragraph can be wrapped as convenient.

Everything that is not plain text starts with `|`. What follows the bar defines the construct:

- a letter — a command: `|toc`, `|table(…)`;
- `(` — the value of an expression: `|(організація)`;
- a punctuation mark — a short form: `|<` heading, `|>` clause, `|*{…}` bold;
- a second bar — the bar itself: `||`.

Inline formatting (the placeholder in this example gives the expected warning P2635):

```
Строк дії — |*{до 31.12.2026}, умови — |/{див. додаток}, код |`{UA-17}.
```

```
Назва документа в лапках мови документа: |"{Положення про закупівлі}.
Незаповнене місце в чернетці: |?{прізвище}.
```

Quotes `|"{…}` follow the document language («…» for Ukrainian). A placeholder `|?{…}` is highlighted and gives a warning in the check, so a draft does not reach a release unnoticed (`--strict` turns the warning into an error).

1.4. Headings, clauses, lists

```
|<(number=False) Положення про відрядження
|< Загальні положення
|> Положення визначає порядок направлення працівників у відрядження.
|> Строк відрядження не перевищує 30 календарних днів.
|>> Строк може бути продовжено наказом керівника.
|< Документи
Працівник отримує:
|- посвідчення про відрядження;
|- кошторис витрат.
```

- `|<` is a level 1 heading, `|<<` — level 2, and so on up to six. The translator sets the numbers; `number=False` — a heading without a number (the title of the document).
- `|>` is a clause, `|>>` — a subclause. Clauses are numbered within a section: “1.1.”, “1.2.”, “1.2.1.” (the styles set decides the look of the numbers: in a report after DSTU 3008 there is no final period).
- `|-` is a bulleted list item, `|#` — a numbered one; `|--`, `|##` — the next level.

A clause, a heading and a list item are line forms: they end at the end of the line. A clause of several paragraphs is written as an environment and closed with `|end>`:

```
|>:
Перший абзац пункту.

Другий абзац того ж пункту.
|end>
```

A list and a table right after a clause belong to it; an ordinary paragraph after a clause is no longer part of it.

1.5. Check and build

```
primula check довідка.prml          # check without writing a file
primula build довідка.prml          # довідка.odt next to the source
primula build довідка.prml --pdf    # a PDF as well (needs LibreOffice)
```

The check reports errors and warnings with the line and column; what to do with them is on the page “Messages of the translator”. In Studio the same commands are “Check document” (`Ctrl+Shift+Enter`) and “Build document (ODT and PDF)” (`F9`); a click on a message opens its line.

1.6. Highlighting in other editors

`.prml` files are plain text and can be edited in any editor. Highlighting for Kate, KWrite, KDevelop, Midnight Commander (mcedit), Geany and VS Code is in the `editors/` directory of the package: `editors/install.sh` installs it into the user profile (`./install.sh kate vscode` for the chosen editors only), and `editors/README.md` describes what each editor can and cannot show. The word list of the highlighting is built from the reference of the translator, so new commands and functions of the language reach it when the kit is updated.

1.7. Default look

When a document names no styles, the built-in set `std/styles/default` applies: Liberation Sans 11 pt, A4 margins, the look of headings, tables, footnotes. For business documents, letters, instructions and reports the kit has ready sets; one header line names one, for example `styles: [std/styles/business]`. More — on the page “Styles”.

2. Contract

The page shows how to make a contract: data in the header, text with numbered clauses, references to clauses instead of numbers typed by hand, sums in words, requisites of the parties and an appendix. The look is the styles set `std/styles/business`.

2.1. The whole document

```
---
doc:
  title: "Договір про надання послуг"
  identifier: "П-2026/017"
  language: uk-UA
styles: [std/styles/business]
params:
  місто: "м. Харків"
  дата: 2026-10-12
  строк_днів: 5
  виконавець: {назва: "ТОВ «Виконавець»", код: "00000000", особа: "директора Петренка
Петра Петровича", підписант: "П. П. Петренко"}
  замовник: {назва: "ТОВ «Замовник»", код: "11111111", особа: "директора Іваненка
Івана Івановича", підписант: "І. І. Іваненко"}
---
|code:
послуги = [
    record(назва="Налаштування обладнання", кількість=dec("2"), ціна=dec("1500.00")),
    record(назва="Навчання персоналу", кількість=dec("1"), ціна=dec("2400.00")),
]
сума = sum([п.кількість * п.ціна for п in послуги])
|end
|.DocumentTitle: ДОГОВІР № |(doc.identifier)
|.DocumentSubtitle: про надання послуг
|.DatePlace: |(місто), |(fmt_date(дата, "contract"))

|.Preamble: |(виконавець.назва) (далі — |.DefinedTerm{Виконавець}) в особі |
(виконавець.особа) та |(замовник.назва) (далі — |.DefinedTerm{Замовник}) в особі |
(замовник.особа) уклали цей договір про таке.

|<(id="предмет") Предмет договору
|>(id="послуги") Виконавець надає послуги, перелічені в |@("специфікація", "label"), а
Замовник приймає й оплачує їх.

|<(id="ціна") Ціна і розрахунки
|> Ціна послуг становить |(money_full(сума, "UAH")).
|>(id="оплата") Замовник оплачує послуги протягом |(строк_днів) |(plural(строк_днів,
"банківського дня", "банківських днів", "банківських днів"))|footnote{Банківський день
— день, у який банки України здійснюють розрахунки.} з дати підписання акта.

|<(id="відповідальність") Відповідальність сторін
|> За прострочення оплати (|@("оплата", "label")) Замовник сплачує пеню в розмірі
подвійної облікової ставки НБУ.

|<(number=False) Реквізити і підписи сторін
|table(style="TableParties", header=1):
|: Виконавець |& Замовник
|: |(виконавець.назва) |& |(замовник.назва)
|: Код ЄДРПОУ |(виконавець.код) |& Код ЄДРПОУ |(замовник.код)
|: _____ |(виконавець.підписант) |& _____ |(замовник.підписант)
|end table

|appendix(id="специфікація", title="Специфікація послуг", subtitle=f"до Договору №
{doc.identifier}"):
|table(style="TableServices", header=1, footer=1, cols=["52%", col("12%", id="к",
type="decimal", align="right"), col("18%", id="ц", type="money", align="right"),
```

```
col("18%", id="c", type="money", currency="UAH", align="right")]):
|: Послуга |& Кількість |& Ціна, грн |& Сума, грн
|for(п in послуги):
|: |(п.назва) |& |(п.кількість) |& |(п.ціна) |& |=(this_row("к") * this_row("ц"))
|endfor
|:|&(span=3, align="right") Разом |& |=(sum(column("c", rows=body_rows())))
|end table
|end appendix
```

Below — part by part.

2.2. Data go into the header, not into the text

Everything that changes from contract to contract goes into `params:` of the header: the parties, the city, the date, the terms. The text shows the values with an expression `|(…)`: `|(місто)`, `|(виконавець.назва)`. A nested record (`виконавець: {назва: …, код: …}`) is read with a dot.

So the text needs no editing when a party changes, and Studio shows the parameters as a separate form. With many contracts, the shared data move into a project (page “Project”).

Do not put real codes, IBANs and signatures into a template — while the data are missing, use a placeholder `|?{…}`: the check reminds of it, and `--strict` does not let a release build.

2.3. Beginning of the contract

The title, subtitle, place and date, preamble are paragraphs with styles of the `business` set: `|.DocumentTitle: …`, `|.DatePlace: …`, `|.Preamble: …`. A style written at the start of a line as `|.Name:` applies to the whole paragraph of that line; `|.Name{…}` — to a fragment inside a line (here `|.DefinedTerm{Виконавець}` marks a defined term).

The contract date — the function `fmt_date(дата, "contract")`: «12» жовтня 2026 р. Other date formats, and also `fmt_date(дата, "long")` and `"short"`, are in the appendix “Functions of expressions”.

2.4. Sections and clauses

`|<` is a section, `|>` — a clause, `|>>` — a subclause. The translator sets the numbers, so a new clause inserted in the middle needs no manual renumbering.

2.5. References to clauses

Never type the number of a clause. The clause or section referred to gets an `id=`, and the text has a reference:

Form	Text in the document
`|@("оплата")`	2.2
`|@("оплата", "label")`	п. 2.2
`|@("ціна", "label")`	розд. 2
`|@("специфікація", "label")`	дод. 1
`|@("ціна", "title")`	Ціна і розрахунки

A reference may come before its target. If a clause is removed or its `id=` changed, the check reports an unknown target — a document never has dangling references. A label in a grammatical case — `|@("ціна", "label", case="loc")` → «розділі 2».

2.6. Sums and numbers

Money in Primula is a decimal number: write `dec("1500.00")` or a header parameter, not a binary float. Output it with library functions:

```
|code:
сума = dec("5400.00")
|end
|(money(сума, "UAH")) — |(money_full(сума, "UAH")).
ПДВ 20 %: |(money(vat(сума, 20), "UAH")), разом з ПДВ |(money(with_vat(сума, 20),
"UAH")).
Строк — |(words(5)) |(plural(5, "день", "дні", "днів")).
```

`money` gives “5 400,00 грн”, `money_full` — the sum in figures and in words, `words` — a number in words, `plural` — the word form for a number. A number or date written out without a formatting function gives a warning, so “5400.0” does not get into a contract.

2.7. Requisites and signatures

The requisites of the parties are a table without lines, `TableParties` (two columns of 50 %): the first row — the names of the parties, then a row per requisite. The signature row is in the same table, so the signatures of the parties stand on one level.

2.8. Appendix

`|appendix(…):` … `|end appendix` starts an appendix on a new page, writes “Додаток 1”, the subtitle and the title, and opens its own numbering of sections, figures and tables. The reference `|@("специфікація", "label")` gives “дод. 1”. The table with a total in the appendix is explained on the page “Act and tables with totals”.

2.9. Footnotes

`|footnote{text}` puts a footnote at the bottom of the page; Writer counts the number.

2.10. Repeated blocks

If a block of requisites or a signature repeats in many contracts, it becomes a macro in a module (`|def`, page “Project”) or a fragment (`|include`). An example from the kit — `examples/02-contract` (the module `lib/business.m.prml` with macros for requisites and signatures).

3. Act and tables with totals

A table is an environment `|table(…):` … `|end table`. A row starts with `|:`, cells are separated by `|&`. The translator computes totals by formulas, checks the number of cells in rows and formats numbers and sums by the document language.

3.1. A simple table

```
|table(cols=["40%", "*"], header=1):
|: Показник |& Значення
|: Строк поставки |& 10 робочих днів
|: Місце поставки |& склад Покупця
|end table
```

- `header=1` — one header row; it repeats on the following pages.
- `cols` — column widths: `"40mm"` — fixed, `"30%"` — a share of the table, `"*"` — the rest, `"auto"` — by content, `"20mm..60mm"` — by content within limits.
- The number of cells of every row must match the number of columns; an extra or missing cell is an error with the row number. An empty cell is `|&` without text.

3.2. Act: rows from data and a total

Let the items of the act lie in a CSV file:

```
назва,кількість,ціна
Налаштування обладнання,2,1500.00
Навчання персоналу,1,2400.00
Виїзд спеціаліста,3,350.00
```

The table of the act with a total and VAT:

```
|code:
позиції = load_csv("data/позиції.csv", types={"кількість": "decimal", "ціна":
"money"})
|end
|table(id="т-акт", caption="Перелік наданих послуг", header=1, footer=3, cols=["10mm",
"*", col("18mm", id="к", type="decimal", align="right"), col("26mm", id="ц",
type="money", align="right"), col("30mm", id="с", type="money", currency="UAH",
align="right")]):
|: № |& Послуга |& К-сть |& Ціна |& Сума
|for(н, п in enumerate(позиції, 1)):
|: |(н) |& |(п.назва) |& |(п.кількість) |& |(п.ціна) |& |=(this_row("к") *
this_row("ц"))
|endfor
|:|&(span=4, align="right") Разом без ПДВ |& |=(sum(column("с", rows=body_rows())))
|:|&(span=4, align="right") ПДВ 20 % |& |=(round(cell(len(позиції) + 2, "с") *
dec("0.20"), 2))
|:|&(span=4, align="right") |*{Усього} |& |=(cell(len(позиції) + 2, "с") +
cell(len(позиції) + 3, "с"))
|end table
Див. |@("т-акт", "label").
```

What happens here:

- `load_csv` reads the file; `types=` declares the column types, otherwise everything is read as strings. The path is relative to the document and does not leave the project root. `load_yaml` and `load_json` work the same way.

- `col(width, id=…, type=…, currency=…, align=…)` describes a column. `id` is the name of the column in formulas. `type="money"` shows the value with two decimals (“1 500,00”), `currency="UAH"` adds “грн”, `type="decimal"` — a number with digit groups.
- `|for(…):` … `|endfor` repeats the row for every item.
- `|=(…)` is a cell formula. It is computed when all ordinary cells are known: `this_row("к")` — the value of column `к` in this row, `column("с", rows=body_rows())` — the column over the body rows (without header and totals), `cell(row, column)` — a cell by row number (from 1, counting the header) and column.
- `footer=3` — three footer rows; they are not part of `body_rows()`.
- `|:|&(span=4, align="right")` — the arguments of the first cell are written right after `|:`; `span=4` merges four columns.
- `caption=` and `id=` give the caption “Таблиця 1 — Перелік наданих послуг” and the reference target “табл. 1”.

Money is computed with decimal numbers, so kopecks are not lost. Rounding — `round(x, 2)`; the rounding of accounting sums and VAT — the library functions `round_money`, `vat`, `with_vat` (appendix “Functions”).

3.3. A table without a loop

When the rows are whole records, no loop is needed:

```
|code:
склад = [record(назва="Кабель", к=dec("120")), record(назва="Роз'єм", к=dec("40"))]
|end
|table(data=склад, fields=["назва", "к"], titles=["Найменування", "Кількість"],
cols=["*", col("30mm", id="к", type="decimal", align="right")], footer=1):
|:|&(align="right") Разом |& |=(sum(column("к", rows=body_rows())))
|end table
```

3.4. A cell of several paragraphs

A compact row `|:` fits in one source line. A cell with a list or several paragraphs uses the expanded form:

```
|table(cols=["35%", "*"], header=0):
|row:
|cell: Обов'язки Виконавця
|cell:
Виконавець зобов'язується:

|- надати послуги в строк;
|- передати звіт.
|end cell
|end row
|end table
```

Compact and expanded rows can be mixed in one table.

3.5. Look of a table

The look of a table is set by its style: `|table(style="TableCompact", …)`. The styles sets of the kit have ready ones: `TableGrid` (default), `TableCompact`, `TableNoBorders`; `business` adds

`TableServices`, `TableSchedule`, `TableForm`, `TableParties` with set column widths (the table's own `cols` replace them). One-off differences are arguments of the table, the row (`|:(background="#EEEEEE")`) and the cell (`|&(align="center")`): table style → column → row → cell, each level refines the previous one. Lines — `frame=` (outer frame) and `rules=` (`"all"`, `"rows"`, `"cols"`, `"none"`).

A table inside a section with columns (`|section(cols=2)`) takes the width of a column.

4. Official letter

A letter uses the styles set `std/styles/letter`. It builds on the business set `business` and follows DSTU 4163:2020: margins — left 30 mm, right 10 mm, top and bottom 20 mm; text with a paragraph indent of 10 mm, the addressee as a block 90 mm from the left margin, the page number from the second page at the top in the middle.

4.1. Example

```
---
doc:
  title: "Щодо графіка технічної підтримки"
  language: uk-UA
styles: [std/styles/letter]
params:
  номер: "12/3"
  дата: 2026-10-16
  відправник: {назва: "ТОВ «Виконавець»", адреса: "вул. Сумська, 1, м. Харків, 61000",
телефон: "+380 57 000 00 00", код: "00000000", керівник: "Петро ПЕТРЕНКО"}
  адресат: {посада: "Директору", назва: "ТОВ «Замовник»", особа: "Івану ІВАНЕНКУ"}
---
|.LetterheadName: |(відправник.назва)
|.LetterheadDetails: |(відправник.адреса), тел. |(відправник.телефон)
|.LetterheadDetails: Код ЄДРПОУ |(відправник.код)
|.OutgoingNumber: Вих. № |(номер) від |(fmt_date(дата, "short"))

|.Recipient: |(адресат.посада) |(адресат.назва)
|.Recipient: |(адресат.особа)

|.LetterSubject: |(doc.title)

|.Salutation: Шановний пане Іване!

Повідомляємо, що роботи з налаштування обладнання завершено, акт приймання-передачі
підписано сторонами.

Пропонуємо погодити графік технічної підтримки на листопад — грудень 2026 року. Проєкт
графіка додається.

|.AttachmentLabel: Додаток: проєкт графіка на 1 арк. в 1 прим.

|table(style="TableParties", header=0, cols=["60%", "40%"]):
|: Директор |& |(відправник.керівник)
|end table

|.Executor: Олена Коваленко +380 57 000 00 01
```

4.2. Parts of the letter

Part	Style	Note
name and details of the sender	`LetterheadName`, `LetterheadDetails`	on the left, lines up to 73 mm
number and date	`OutgoingNumber`	“Вих. № … від …”; the date — `fmt_date(дата, "short")`
addressee	`Recipient`	each line is a separate paragraph of this style
heading of the text	`LetterSubject`	“Щодо …”, without a period
salutation	`Salutation`	centred
text	no style	ordinary paragraphs get `BodyIndented` — the set binds the text to it
attachments	`AttachmentLabel`	

Part	Style	Note
signature	table `TableParties`	the position on the left, the name on the right on one line
executor	`Executor`	in small print at the end

The first page of the letter has the page style `LetterFirst` without a number; the following pages are `Portrait` with the number at the top. The set does this; nothing needs writing in the text.

4.3. Your own differences

If the organisation uses another letterhead, do not edit the set of the kit: make your own module over it and name it instead of the set (page “Styles”):

```
---
kind: styles
extends: std/styles/letter
paragraphs:
  LetterheadName: {size: 13pt, color: "#1F4E79"}
---

---
doc: {title: "Лист", language: uk-UA}
styles: [styles/бланк.s.prml]
---
|.LetterheadName: ТОВ «Приклад»

Текст листа.
```

A logo on the letterhead — `|img("assets/логотип.svg", style="LogoSmall")` at the beginning of the letter; the sizes of logos and illustrations (`LogoSmall`, `Logo`, `Illustration` …) are in the `business` set.

5. Technical documentation

Manuals, instructions and technical descriptions use the styles set `std/styles/technical` (or `report` — for reports after DSTU 3008). Below are the means that technical documents need: admonitions, variants for different versions of a product, abbreviations, an index, figures.

5.1. Admonitions and notes

`|!("kind")` is semantic markup: the author names the kind, and the signal word in the document language, the colour and the look come from the styles.

Kind	Word (uk)	When
`danger`	НЕБЕЗПЕКА	imminent danger to life
`warning`	ПОПЕРЕДЖЕННЯ	possible danger to life or serious injury
`caution`	ОБЕРЕЖНО	possible minor injury
`notice`	УВАГА	possible damage to property
`note`	Примітка	explanation
`tip`	Порада	recommendation

```
|!("warning", hazard="Висока напруга", consequence="Ураження електричним струмом"):
Перед обслуговуванням від'єднайте пристрій від мережі.

|- Вимкніть автоматичний вимикач.
|- Перевірте відсутність напруги.
|end!

|!("notice") Не встановлюйте пристрій поблизу джерел тепла.

|!("note") Значення наведено для температури 20 °C.
|!("note") Допуск — ±5 %.
```

- The line form `|!("kind") text` is a block of one paragraph; the environment `|!(…):` … `|end!` — of several paragraphs and lists.
- `danger` and `warning` need `hazard=` (the kind and source of the hazard); without it the check warns.
- Several `note` blocks in a row form a group “Примітки:” with numbers.

5.2. Variants for different versions

One source text serves several versions of a product. The properties of a version are listed in the parameter `features`, and conditional parts are marked with `when=`:

```
---
doc: {title: "Настанова з експлуатації", language: uk-UA}
styles: [std/styles/technical]
params:
  модель: "X300"
  features: [wifi]
---
|< Підключення
|> Під'єднайте кабель живлення.
|>(when=has("wifi")) Підключіть пристрій до мережі Wi-Fi.
|>(when=has("bluetooth")) Увімкніть Bluetooth на телефоні.
|table(header=1, cols=["*", "30%"]):
|: Параметр |& Значення
|: Напруга |& 230|~В
```

```
|:(when=модель == "X300") Потужність |& 2,2|~кВт
|end table
|if(has("wifi")):
|< Бездротова мережа
|> Пристрій підтримує стандарт 802.11n.
|endif
```

- `when=` exists on any block command, environment and table row; `|if(…):` … `|endif` — for any group of blocks.
- `has("wifi")` is true when `wifi` is in `features`. The set of properties comes from the document header, the project or a build target of the project (`params: {features: [wifi, bluetooth]}` in `targets`): so the manuals of all versions are built from one text. The command line parameter `--param` does not take lists.
- Removed parts do not reach the ODT; numbering, the table of contents and references are computed over the remaining text. A reference to a removed part is an error.
- `primula build настанова.prml --review` builds all variants at once and marks every conditional part with its condition (green — included in the current version, red — not), and puts the watermark “НЕ ДЛЯ ВИПУСКУ” on the pages. So the whole text is reviewed in one file.

5.3. Abbreviations

```
|abbr("ПЗ", "програмне забезпечення") встановлюється з диска.
Оновлення |abbr("ПЗ") виконується автоматично.
|list_of("abbr", title="ПЕРЕЛІК СКОРОЧЕНЬ")
```

At the first use the output is “програмне забезпечення (ПЗ)”, after that only “ПЗ”. The case of the expansion at this place — `text="програмного забезпечення"`. `|list_of("abbr")` builds the list of abbreviations as a table.

5.4. Index, table of contents, lists

```
|toc(max_level=2, title="ЗМІСТ")
|< Живлення
Напруга|index("напруга живлення") подається через адаптер.
|list_of("index", title="ПОКАЖЧИК")
```

The table of contents, the lists of figures and tables (`|list_of("figure")`, `|list_of("table")`) and the alphabetical index are indexes that Writer updates; Writer fills their page numbers, or the translator when it builds a PDF.

5.5. Figures

```
|figure(svg='<svg xmlns="http://www.w3.org/2000/svg" width="40mm" height="20mm"><rect
x="1" y="1" width="38" height="18" fill="none" stroke="black"/></svg>', id="рис-
схема"):
Схема підключення
|end figure
Див. |@("рис-схема", "label").
```

- `|figure(path or svg=…, id=…):` … `|end figure` — a numbered figure; the body is the caption (“Рисунок 1 — Схема підключення”). `|img(…)` — an image without number and caption.
- A file path is relative to the document: `|figure("assets/схема.svg", width="120mm")`. Formats — PNG, JPEG, SVG.

- `svg=` — an image from an SVG string computed by code (a chart, a scale, a QR code from a module function).
- Size — `width=` or `height=`; sizes by style — `style="Illustration"` (sets `business`, `letter`).

5.6. Program text and interface

The `technical` set has styles for interface elements and commands:

```
---
doc: {title: "Інструкція", language: uk-UA}
styles: [std/styles/technical]
---
|list("StepList"):
|# Натисніть |.Key{Ctrl+S}.
|# Виберіть |.MenuItem{Файл → Зберегти} і вкажіть файл |.FileName{звіт.odt}.
|end list
|.StepResult: Файл буде збережено.

|.Preformatted: primula check звіт.prml
```

Character styles `Key`, `MenuItem`, `FileName`, `Command`, `Variable`; paragraph styles `Preformatted`, `ProgramOutput`, `StepResult`, `DefinitionTerm`/`DefinitionText`; the step list `StepList`; the table `TableRevisions` for a record of revisions. The full list — appendix “Styles of the kit”. Text that must be kept letter for letter (a listing with `|` characters) goes into `|verbatim:` … `|end`.

5.7. Shared fragments and keys

Repeated texts (general safety measures, warranty) go into fragments `*.f.prml` and are inserted with `|include`; in a project they can be swapped per version through the key map `keys:` — page “Project”. Complete examples — `examples/03-manual` and `examples/06-safety`.

6. Project of several documents

When there are several documents — a contract, acts, invoices, letters of one company — the shared data, styles and repeated texts are collected in a project. A project is a directory with the file `project.p.prml`. The translator finds it itself, going up from the document (up to a directory with `.git`).

```
договори-2026/
  project.p.prml        parameters, styles, keys, build targets
  договір.prml
  акт.prml
  fragments/            texts for |include (*.f.prml)
  lib/                  macros and functions for |import (*.m.prml)
  styles/               your own styles module (*.s.prml)
  data/                 CSV, YAML, JSON
```

6.1. Project file

```
---
kind: project
name: "Договори 2026"
language: uk-UA
styles: [styles/компанія.s.prml]
doc_defaults: {organization: "ТОВ «Виконавець»", creator: "Відділ договорів"}
params:
  місто: "м. Харків"
  виконавець: {назва: "ТОВ «Виконавець»", код: "00000000", директор: "П. П. Петренко"}
  ставка_пдв: 20
targets:
  - {source: договір.prml, output: out/договір.odt, pdf: true}
---
```

- `params:` — shared parameters. A document sees them by name (`|(місто)`) and as `|(project.місто)`. A document parameter with the same name replaces the project value for that document.
- `doc_defaults:` — default ODT metadata for all documents; the `doc:` key of a document replaces them one by one.
- `styles:` — styles modules of the project; they apply to all documents.
- `targets:` — targets of the batch build.

The project's own styles module builds on a set of the kit (page “Styles”):

```
---
kind: styles
extends: std/styles/business
paragraphs:
  Реквізити: {parent: Details, keep_together: true}
---
```

6.2. A document of the project

```
---
doc: {title: "Договір про надання послуг"}
---
|(виконавець.назва), |(місто). Ставка ПДВ — |(percent(ставка_пдв)).

|.Реквізити: Код ЄДРПОУ |(виконавець.код)
```

The document names neither styles nor a language: everything comes from the project.

6.3. Fragments: shared text

A fragment `*.f.prml` is a piece of text to insert. The parameters of a fragment are declared in its header and passed as arguments of `|include`:

```
---
params: {місяців: 12}
---
|> Гарантійний строк становить |(місяців) |(plural(місяців, "місяць", "місяці",
"місяців")).

|< Гарантія
|include("fragments/гарантія.f.prml", місяців=24)
```

- The path is relative to the file with the command; the extension may be omitted (`fragments/гарантія`).
- Names defined in a fragment are not visible to the document; to pass a value out, the fragment exports it: `|export(строк=…)`, a function — with the decorator `@export` above `def`, a ready macro — `|export("підпис")` (page "Code").
- A part of a fragment is marked `|part("назва"):` … `|end part` and inserted with `|include("file", part="назва")`.

6.4. Modules: macros and functions

A module `*.m.prml` holds only definitions — macros `|def`, functions and data in `|code:`. It has no text.

```
|def підпис(сторона, особа):
|.SignatureRole: |(сторона)
|.SignatureLine: _____________ |(особа)
|enddef

|import("lib/підписи")
|підписи.підпис("Виконавець", виконавець.директор)
```

After `|import("lib/підписи")` the definitions are available as `підписи.name`; `|import("lib/підписи", "підпис")` — without the prefix, `alias="п"` — under a short name. Nothing needs to be exported from a module: all names except those starting with `_` are public. An `|import` of the document also works in its fragments. A module is evaluated once per build.

6.4.1. A library shared by several projects

An `|import` path does not leave the project root, but a symbolic link created inside the project may lead to a shared directory (decision 136):

```
ln -s ../shared-lib lib     # in the project root
```

Then `|import("lib/ціни")` takes the module from `../shared-lib`. The build gives warning P2639 with the name of the link and its target: a directory outside the project is not in the source archive, so keep it together with the projects (or in the same repository). Paths inside the library are counted from `lib/…`, and `../` in the text still does not leave the root. In the report and messages the file is written as `lib/ціни.m.prml`.

6.5. Data from files

```
номер,дата,сума
1,2026-10-31,5400.00
2,2026-11-30,5400.00

|code:
акти = load_csv("data/акти.csv", types={"номер": "decimal", "дата": "datetime",
"сума": "money"})
|end
|for(a in акти):
|- Акт № |(a.номер) від |(fmt_date(a.дата, "short")) на |(money(a.сума, "UAH")).
|endfor
```

Without `types=` all CSV values are strings (codes with leading zeros stay intact). YAML and JSON are read by `load_yaml` and `load_json`; mappings become records with dot access.

Spreadsheets of LibreOffice Calc and Excel are read by `load_ods` and `load_xlsx` (decision 138): the sheet — `sheet="Позиції"` or a number from 1, the range — `range="A1:D20"`, the first row — field names (`header=False` — fields `A`, `B`, …), `types` — as for CSV.

```
|code:
позиції = load_ods("data/позиції.ods", sheet="Позиції", types={"ціна": "money"})
|end
```

Formulas are not computed: the result saved in the file is taken — save the spreadsheet in Calc or Excel after editing it. Numbers are `decimal` of up to 15 significant digits, dates are `datetime`; an error in a cell (`#DIV/0!`) or a formula without a result stops the build with the name of the sheet and the cell. Save `.xls` files as `.xlsx` or `.ods` first.

`load_text` returns the whole text file as one string (decision 140): UTF-8, line ends LF, no parsing. It keeps the SVG of a picture, a sign or a diagram outside the document, to be put in with `svg=`:

```
|img(svg=load_text("img/знак.svg"), width="12mm")
```

6.6. Building the project

```
primula build project.p.prml --pdf      # all targets of the project
primula check project.p.prml            # check all targets without writing
primula build договір.prml              # one document (with the project's parameters
and styles)
primula check project.p.prml --dump-project   # the project description as JSON
```

Targets can be made by code: the code section of the project file (after the header) reads data and calls `build()` — for example, an act per CSV row:

```
---
kind: project
---
for row in load_csv("data/акти.csv", types={"номер": "decimal", "сума": "money"}):
    build("акт.prml", params={"номер": row.номер, "сума": row.сума}, output=f"out/акт-
{row.номер}.odt", pdf=True)
```

An error in one target does not stop the others. A complete example with a contract, acts from CSV, a key map and an assembled package — `examples/07-project`.

6.7. Export to Markdown, HTML, DOCX

`primula export` writes the evaluated document to other formats through pandoc: numbers of clauses, references, notes, tables and captions are kept, page layout (headers, footers, columns, page numbers) is not.

```
primula export договір.prml                    # договір.md
primula export договір.prml --to html          # one HTML file with the images
primula export договір.prml --to docx          # paragraph styles by name (custom-
style)
primula export договір.prml --to json          # the pandoc AST, no pandoc needed
```

6.8. Quick view: primula view

`primula view` builds the document in a temporary directory (ODT and PDF, as `build --pdf` does), opens the PDF in a viewer and deletes the files when the window is closed. If the build fails, the build log is shown instead of the PDF, in the terminal or in a window. The command is meant for file managers: the short name `primula-view` lies next to `primula`.

```
primula view договір.prml
primula view договір.prml --param номер=12 --viewer "okular --unique"
primula view --text договір.prml | less        # no LibreOffice: plain text through
pandoc, about a second
```

The viewer is taken from `--viewer`, then from the variable `PRIMULA_VIEWER`, then it is the PDF viewer of the desktop (okular, evince, atril, xreader, zathura, mupdf, qpdfview), then `xdg-open`. A viewer that is given or recognised is waited for; `xdg-open` returns at once, so its files stay in `$XDG_RUNTIME_DIR/primula-view` and are deleted later. LibreOffice starts with a profile in `~/.cache/primula`, which speeds up repeated runs a little; most of the time goes to the layout in LibreOffice, so a document of several pages takes a few seconds, and `--text` is for a quick look at the text.

F3 in Midnight Commander goes into `~/.config/mc/mc.ext.ini`:

```
[primula]
Extensions=prml
View=primula-view %f
```

For the text in the terminal itself: `View=%view{ascii} primula view --text %f`.

6.9. Assembled document

Several documents are built into one ODT with the command `|document`:

```
|document("договір.prml", scope="договір")
|document("акт.prml", scope="акт1", params={"номер": 1})
```

Every part starts on a new page, is numbered in its own scope and evaluated separately; the table of contents and the page numbering are shared. A reference into another part uses the full ID: `|@("акт1:перелік")`.

6.10. References to another document

An act can refer to a clause of the contract when the contract is named in `ref_documents:` of the project and the act lists it in `references:` of its header: `|@("оплата", "label")` in the act gives “п. 2.2” of the contract. The text of the reference comes from the registry of the contract, which the translator updates when building.

6.11. Key map

The `keys:` of a project give names to resources and fragments: `|img(key="логотип")`, `|include(key="умови.оплата")`. A build target can override a key (`keys:` of the target) — so one text is built with different logos or variants of terms.

6.12. Order project

For a series of orders under one contract, a project is made in a subdirectory with `extends: ../../project.p.prml`: it inherits the parameters, styles, keys and documents of the base project and adds the data of the order. In Studio — “File → New → Order project”.

7. Styles

The look of a document is set by styles, not by the text: font, indents, spacing, table frames, page margins. The text says only *what* something is — a heading, a clause, requisites — and the styles module decides *how* it looks. So the same document can be printed differently by naming another set, and changing the look does not touch the text.

7.1. Choosing a set

The kit of the translator has styles sets for typical documents:

Set	For	Base
`std/styles/default`	applies by itself when no styles are named	—
`std/styles/business`	contracts, acts, invoices, protocols; DSTU 4163:2020	`default`
`std/styles/letter`	official letters; DSTU 4163:2020	`business`
`std/styles/technical`	manuals, instructions, technical descriptions	`default`
`std/styles/report`	reports in science and technology; DSTU 3008:2015	`default`
`std/styles/compact`	memos, quick references, cheat sheets	`default`

A document names a set with one header line:

```
---
doc: {title: "Акт", language: uk-UA}
styles: [std/styles/business]
---
|.DocumentTitle: АКТ № 14
```

The base (`default`) need not be listed: the set names it itself. The styles of every set, their properties and the default font are in the appendix “Styles of the kit”; samples — the documents `examples/08-styles` (build them and look at the PDF).

7.2. Applying styles in the text

Form	What it does
`|.DatePlace: м. Харків`	a paragraph style for the paragraph of this line
`|.Note:` … `|end.Note`	a paragraph style for several paragraphs (environment)
`|.DefinedTerm{Замовник}`	a character style for a fragment inside a line
`|*{…}`, `|/{…}`, `|`{…}`	bold, italic, monospace — character styles too (`Strong`, `Emphasis`, `Code`)
`|table(style="TableCompact")`	a table style
`|list("DashList")`	the style of the next list

The style name in `|.Name` is an identifier: it is not translated and does not change when the text is proofread. An unknown name is an error with its position.

Headings, clauses, lists, tables and captions get their styles automatically — the binding of constructs in the styles module sets them (`Heading1`…`Heading6`, `Clause`, `BulletList`, `TableGrid`, `TableCaption` …). The command `|set` changes the style of a construct until the end of the enclosing environment:

```
|set(list="DashList")
|- перший пункт;
|- другий пункт.
```

Direct formatting `|fmt(bold=True){…}` exists too, but gives a warning: one-off formatting is lost when the set changes, and equal places end up formatted differently. If a format repeats, make a style.

7.3. Your own styles module

Do not edit the sets of the kit — make your own module over a set. It holds only the differences and names its base with the key `extends`:

```
---
kind: styles
extends: std/styles/business
defaults: {font: Roboto, size: 11pt}
paragraphs:
  Body:        {line_height: "120%"}
  Реквізити:   {parent: Details, keep_together: true}
characters:
  Акцент:      {bold: true, color: "#1F4E79"}
tables:
  TableServices: {header_background: "#DDE6F0"}
---

---
doc: {title: "Договір", language: uk-UA}
styles: [styles/компанія.s.prml]
---
Текст з |.Акцент{акцентом}.

|.Реквізити: Код ЄДРПОУ 00000000
```

- The document (or the project) names only your module: the base and its base (`business` → `default`) are loaded by themselves.
- A style property under the same name adds to the style of the set: `Body: {line_height: "120%"}` changes only the spacing, the rest stays from the set.
- A new style inherits an existing one through `parent`.
- The font of the whole document changes with one line `defaults: {font: …}`. The font must be installed where the document is opened or the PDF is built.

Style groups of a module: `paragraphs` (paragraph styles), `characters` (character styles), `tables`, `lists`, `blocks`, `frames` (size and placement of images), `pages` (pages). Properties — `font`, `size`, `bold`, `align`, `line_height`, `space_before`, `space_after`, `left`, `first_line`, `keep_with_next` and others; the full list is section 11.3 of the specification. Studio gives hints on properties and styles (mouse hover, `Ctrl+Space`).

7.4. Layers

When styles are named in several places, they are applied in layers, from early to late:

1. modules from `styles:` of the project;
2. modules given at build time with `--style FILE`;
3. modules from `styles:` of the document;
4. styles defined by the code of the document itself.

A later layer adds to and overrides the earlier ones. The base of a module (`extends`) is the layer right before it. Which layers are actually loaded and where each style comes from is shown by the “Styles” panel of Studio or by `primula check документ.prml --dump-styles`.

7.5. Pages, headers and footers

Margins and page size are a page style (`pages:` of the module, `Portrait` by default; the sets have `Landscape`). Headers and footers are set in the code of a styles module or in the document:

```
|footer("Portrait"):
|.Footer: Договір № П-2026/017 — сторінка |field.page з |field.pages
|end footer
```

A page change from this place — `|page(style="Landscape")`, a page break — `|newpage`.

7.6. Moving from ODT Markdown Kit

The business set of Kit 1.2 (`business-styles.yaml`) is carried over to `std/styles/business` and `default` under the same names, except the ones below:

Kit	Primula
`TocTitle`, `TocEntry`	`ContentsTitle`, `Contents1`…`Contents6`
`StrongText`, `EmphasisText`, `UnderlinedText`	`Strong`, `Emphasis`, `Underline` (short forms `|*{}`, `|/{}`, `|_{}`)
`DraftPlaceholder`	`Placeholder` (short form `|?{…}`)
`BodyRoboto`, `FontCarlito` and similar	none: the font is set by `defaults: {font: …}` of your module
`Recipient`, `LetterSubject`, `Salutation`	in the `letter` set
`Approval`	in `business`, 100 mm from the left margin (DSTU 4163:2020)
`NumberedList` with letters `a)`	`NumberedList` (`1.`, `1.`, `a)`), `LetterList` — letters of the document language «a)», `LatinLetterList` — «a)»
image presets `Logo`, `Illustration` …	frame styles with the same names: `|img(…, style="Logo")`

Kit wrote clause numbers and references as text; in Primula the translator sets them (`|>`, `|@("id")`), and the style `Clause` is applied to clauses by itself.

8. Messages of the translator

The translator checks the whole document — from the encoding to references and tables — and reports everything it finds, not only the first error. A document with errors is not built; with warnings it is built, but a warning almost always points to a real problem.

8.1. How to read a message

```
договір.prml:6:9: error[P2201]: name 'сумма' is not defined
 6 | Сума: |(сумма).
   |         ^^^^^
договір.prml: 1 error, 0 warnings
```

- `договір.prml:6:9` — file, line, column. An error in a fragment or a module names its file and the place where it is included.
- `error` — an error, `warning` — a warning.
- `P2201` — the code. The first digit is the area: P0 command line and files, P1 syntax, P2 names and evaluation, P3 styles, P4 structure and references, P5 tables, P6 pages and graphics, P7 modules and project, P8 writing the ODT, P9 technical documentation. The full list with the texts — appendix “Messages”.
- The line with `^^^` shows the place; lines `= help:` and `= note:` are a hint and an explanation.

The language of messages — `--lang uk|ru|en` or the variable `PRIMULA_LANG`. For programs and editors — `--format json`: a list of messages with the fields `code`, `severity`, `message`, `file`, `line`, `column`. Exit code: 0 — no errors, 2 — there are errors.

In Studio the messages of a check are collected in a panel; a click opens the place in the editor, F1 on a construct — its specification section.

8.2. Warnings and the release

`--strict` turns warnings into errors with the same code. This is how the final version is built: unfilled places `|?{…}`, unformatted numbers and direct formatting do not pass.

```
primula build договір.prml --strict --pdf
```

8.3. Common errors

8.3.1. A style without text — P1209

The style name after `|.` is an identifier, not text. If it changed during editing (“Місце-дата” → “Місце і дата”), the line is no longer a style:

```
|.Місце-дата
```

Restore the style name and write the text after the colon: `|.Місце-дата: м. Харків, 12.10.2026`. The list of style names — the “Styles” panel of Studio or `--dump-styles`.

8.3.2. Name not defined — P2201

```
Сума: |(сумма).
```

A typo in the name of a parameter or variable, a parameter not declared in `params:` of the document or the project, a module function without `|import`. In a document the module `std/math` is imported explicitly: `|import("std/math")`.

8.3.3. A command “swallowed” a word — P2601

```
|(1)|sellerом
```

A letter right after a command name continues the name: `|sellerом` is the command `sellerом`. The separator `|;` ends the name: `|seller|;ом`.

8.3.4. A space after the bar — P1201

```
Ціна | 100 грн.
```

After `|` there must be a letter, `(` or the sign of a short form. The bar itself is written in text as two bars: `||`.

8.3.5. Wrong closing — P1218, P1219, P1224

`|end` closes the nearest environment, but not control constructs: `|if` is closed by `|endif`, `|for` — `|endfor`, `|def` — `|enddef`. A block `|code:` is closed by a line with a bare `|end`, without a name (`|end code` is the error P1224). A clause in the line form (`|> text`) is not an environment, and `|end` after it is superfluous (P1218).

```
|> Пункт.
|end
```

8.3.6. A value cannot be written out — P2603, P2604

`|(…)` writes out a string, a number, a date or markup. `None`, a boolean, a list or a record is the error P2603: use `str()`, `join_list()` or a field of the record.

A fractional number, a sum or a date without a formatting function is the warning P2604:

```
|(dec("1.5"))
```

Format with the library: `num(x, 2)`, `money(x, "UAH")`, `fmt_date(d, "long")`.

8.3.7. Unknown style — P3108

```
|.Невідомий: текст
```

No loaded module has the style: a typo or the set is not named (`styles:`). Which modules are loaded — `--dump-styles` or the “Styles” panel.

8.3.8. Reference and ID — P4301, P4102

```
Див. |@("оплата").
```

No target has this `id=`: a typo, or the target is removed or excluded by a condition. A repeated `id=` is the error P4102 with the positions of both places.

8.3.9. A skipped level — P4201

```
|< Розділ
|<<< Підрозділ
```

Headings go level by level: after `|<` comes `|<<`, not `|<<<`. The same holds for lists and clauses.

8.3.10. Table — P5103, P1226

```
|table(cols=["*", "*"]):
|: а |& б |& в
|end table
```

The number of cells of a row does not match the number of columns (counting `span=`). An empty cell is written explicitly: `|&` without text. Text inside a table outside a row `|:` is the error P1226.

8.3.11. A header key was renamed — P1120

The keys `document:` and `documents:` of earlier versions were renamed: metadata — `doc:`, in a project — `doc_defaults:` and `ref_documents:`.

8.3.12. Placeholder — P2635

`|?{…}` was left in the document. In a draft build it is a warning, with `--strict` — an error.

8.4. When it is not clear what is wrong

- Check the piece separately: `primula console документ.prml` opens the evaluator console with the names of the document — an expression can be evaluated there (`:help name` — the description of a function).
- `primula check документ.prml --dump-tree` shows how the translator understood the structure: where a clause ends, what went into a table.
- In Studio the right mouse button on a construct gives a hint and the specification section.

9. Parallel texts

The environment `|parallel` puts two or more texts — tracks — side by side and aligns horizontally the starts of their blocks of meaning — segments. This is how a bilingual agreement, a protocol of disagreements ("old / new") or a text with a comment in the margin are written. Columns of a section (`|section(cols=2)`) do not fit: they flow independently. A table looks right, but it has no notion of "a clause and its translation": numbering, references and language are common to the whole table.

The translator writes `|parallel` as a table without borders, but it knows the tracks: every track is numbered with its own counters, evaluated in its own language and compared with the primary track.

9.1. Example

```
---
doc: {title: "Договір / Agreement", identifier: "ДП-2026/017", language: uk-UA}
params:
  ціна: 48000.00
  виконавець: {uk: "ТОВ «Виконавець»", en: "Vykonavets LLC"}
styles: [std/styles/business]
---
|parallel(tracks=[track("uk", lang="uk", title="Українською мовою"), track("en",
lang="en", title="In English")],
          gap="8mm", rule="0.3pt solid #999999", parity="translation"):

|segment(span="all"):
|.DocumentTitle: ДОГОВІР № |(doc.identifier) / AGREEMENT No. |(doc.identifier)
|end segment

|segment(id="subject"):
|track("uk"):
|<(id="subject") Предмет договору
|>(id="services") |(getattr(виконавець, track.id)) надає послуги з розроблення
документації.
|> Ціна послуг — |(money_full(ціна, "UAH")) (див. |@("services", "label")).
|track("en"):
|<(id="subject") Subject of the Agreement
|>(id="services") |(getattr(виконавець, track.id)) provides the services of developing
documentation.
|> The price of the services is |(money_full(ціна, "UAH")) (see |@("services",
"label")).
|end segment

|end parallel
```

In the Ukrainian column the sum reads «сорок вісім тисяч гривень 00 копійок» and the reference «п. 1.1»; in the English one — "forty-eight thousand hryvnias 00 kopiykas" and "cl. 1.1". Clauses are numbered alike in both columns.

9.2. Tracks

A track is described by the function `track(id, …)` in the argument `tracks=`:

Argument	What it sets
`lang`	language of the track: sums in words, dates, quotes, labels of references, spell checking in Writer
`title`	title above the column; the row of titles is repeated on every page
`width`	width of the column, as for a table column: `"30%"`, `"60mm"`, `"*"`

Argument	What it sets
`primary=True`	the primary track (the first by default): the numbering of the document, the table of contents and the navigation follow it, the other tracks are compared with it
`numbered=False`	a track without numbering: comments, notes in the margin
`optional=True`	the track may have no block in a segment — the cell stays empty
`block`	a block style for all cells of the track (background, padding)

When only languages are needed, the short form `langs=["uk", "en"]` is enough. The gap between the columns is `gap=`, the line — `rule=`.

Parameters of the document are common to all tracks. A value in the language of the track is chosen by code: a track block has the record `track` (`track.id`, `track.lang`), so `|(getattr(виконавець, track.id))` takes the field `uk` or `en`.

9.3. Segments

A segment is a row of alignment: the block of every track starts at the same level, the end follows the longest block. A segment is usually a section of an agreement or a clause.

- **Environment:** `|segment(id="…"):`, then the track blocks `|track("uk"):`, `|track("en"):` and `|end segment`. A track block ends at the next `|track` or at the closing of the segment.
- **Line** — for short blocks:

```
|parallel(langs=["uk", "en"]):
|segment(id="city") |track("uk"){м. Харків} |track("en"){Kharkiv}
|end parallel
```

- **Full width** — text common to all tracks (the title of the agreement, the table of signatures): `|segment(span="all"):` without track blocks.

Inside a track block you write ordinary text: headings, clauses, lists, tables, figures, footnotes. Not allowed: a change of page, sections with columns, appendices, headers and footers, a table of contents, a nested `|parallel` (message P5304). Between segments `|newpage` and `|page(…)` are allowed: the table goes on on a new page with the row of titles.

A heading left last on a page is not moved by Writer to the next segment. Write a heading in the same segment as the first clause of its section.

9.4. Numbering, references, footnotes

- Every track has its own counters: on entering `|parallel` they are copied from the document; after the environment the document goes on with the numbering of the primary track.
- An ID inside a track gets the name of the track: `id="services"` in the Ukrainian block is `uk.services`, in the English one — `en.services`. So equal `id=` in both tracks do not conflict.
- `|@("services")` inside a track looks for the target in its own track, then in the primary one, then in the whole document. A target of another track is written in full: `|@("uk.services")`.
- A footnote of a non-primary track gets the number of its track: the numbers are the same in both columns.
- Headings of non-primary tracks look like headings, but they are not in the Writer navigator and the PDF bookmarks — only the primary track is there.

The table of contents of the document takes the headings of the primary track. The table of contents of another track is `|toc(track="en")`; its default title is in the language of the track ("CONTENTS"):

```
|toc(title="ЗМІСТ")
|toc(track="en")
|parallel(langs=["uk", "en"]):
|segment:
|track("uk"):
|< Предмет договору
|track("en"):
|< Subject of the Agreement
|end segment
|end parallel
```

9.5. Correspondence check

The argument `parity=` compares every numbered track with the primary one in every segment:

Profile	What is checked
`structure` (default)	whether every required track has a block; the sequence of headings, clauses, list items, tables, figures, admonitions
`translation`	the same, and also IDs, targets of references, names of parameters in `|(…)`, numbers and addresses in the text, the counters of the tracks at the end
`none`	nothing

A segment may set its own profile: `|segment(id="idiom", parity="none")` — for a free translation. Differences are warnings P5310–P5315; with `--strict` they become errors.

```
|parallel(langs=["uk", "en"], parity="translation"):
|segment(id="term") |track("uk"){Строк — 5 днів.} |track("en"){The term is 7 days.}
|end parallel
```

9.6. Translation: blanks, confirmation, freshness

When a document is being translated, it helps to keep the state of every block:

1. **Blanks.** `primula parallel договір.prml --stub --track en` writes into the segments without an English block a copy of the structure of the primary block with placeholders `|?{…}`. Placeholders are warnings, with `--strict` errors, so an unfilled translation does not get into a release.
2. **Confirmation.** A checked translation is recorded: `primula parallel договір.prml --stamp --track en --status reviewed` writes `status="reviewed"` and `rev="…/…"` into the blocks — the hashes of the text of the primary block and of the translation itself. `--segment ID` confirms only the named segments.
3. **A changed original.** If the Ukrainian text is changed after the confirmation, `check` gives P5320 "the translation is outdated"; if the translation itself is changed — P5321. The warning stays until the translation is fixed and confirmed again.
4. **Before a release.** `primula parallel договір.prml --check-status reviewed` ends with an error if some block is not in this state or is outdated.

In Studio the same actions are in the menu "Build → Parallel tracks" and in the context menu of the editor ("Confirm the translation of segment …" for the block under the cursor). The Structure panel shows segments and track blocks with their status and marks the outdated ones.

The hash is computed over the text of the block without comments and extra spaces, so reformatting does not make a translation outdated. Expressions `|(…)` enter the hash as written: a changed value of a parameter does not outdate a translation — the values are common to the tracks.

9.7. Protocol of disagreements

Three tracks: the wording of the supplier (optional — a new clause is not in it), the wording of the buyer (the primary track) and a comment of a lawyer (not numbered, a narrow column). The numbers of the clauses of the contract are written explicitly:

```
|parallel(tracks=[track("old", title="Редакція Постачальника", optional=True),
                  track("new", title="Редакція Покупця", primary=True),
                  track("note", title="Коментар", width="30%", numbered=False,
optional=True)]):
|segment(id="p-4-2"):
|track("old"):
|*{4.2.} Оплата здійснюється протягом 5 банківських днів.
|track("new"):
|*{4.2.} Оплата здійснюється протягом 30 календарних днів.
|track("note"):
|.SmallPrint: Відстрочку погоджено листом від 12.10.2026.
|end segment
|segment(id="p-4-3"):
|track("new"):
|*{4.3.} Покупець має право на дострокову оплату.
|end segment
|end parallel
```

Complete examples: `examples/09-parallel/agreement.prml` (a bilingual agreement with confirmed translations) and `protocol.prml`.

9.8. Styles

The output table has the style `ParallelTable` (no borders), the row of track titles — `ParallelHeader`; both are in every styles set of the kit (appendix styles of the kit). Own names are set in a styles module: `structure_styles(parallel_table="…", parallel_header="…")` or by the argument `style=` of `|parallel`.

10. Code: the Python dialect

Primula code is written like Python: expressions `|(…)`, arguments of commands, `|code:` blocks, functions and macros. The syntax is a subset of Python 3.11, but the meaning differs in places: numbers are decimal, global names are immutable, there is no output (`print`). This page lists what an author who knows Python needs to know; what works as in Python is not repeated here.

A complete example with tables "expression → Primula → Python 3" is `examples/10-code` (the document `code.prml`, fragments by topic, the error files `forbidden.prml` and `mistakes.prml`). The example shows the source text: code and definitions are not printed in the PDF, so the PDF is read together with the source. Library functions are in the appendix "Functions of expressions".

10.1. Where code is written

Form	Purpose
`|(сума * 2)`	put a value into the text
`|table(cols=["*", "30mm"])`	arguments of commands are expressions
`|if(n == 1){день}|else{дні}`, `|for(p in позиції){…}`	condition and loop; the body is markup, not code
`|code:` … `|end`	definitions, functions, calculations
`|let(n=5)`	a definition in the middle of the text
`|def підпис(роль):` … `|enddef`	macro: a function whose body is markup

10.2. Numbers: decimal instead of int and float

An integer and a number with a point and no exponent are `decimal`: an exact decimal number of up to 64 significant digits that keeps the number of digits after the point. There is no separate `int`: `type(5)` is `"decimal"`. A binary `float` comes only from a literal with an exponent (`1.5e3`) or `float(x)`; `decimal` together with `float` gives `float`.

Expression	Primula	Python 3
`0.1 + 0.2 == 0.3`	`True`	`False`
`2.20 * 3`	`6.60`	`6.6000000000000005`
`7 / 2`	`3.5` (`decimal`)	`3.5` (`float`)
`round(2.5)`	`3`	`2`
`round(2.675e0, 2)`	`2.68`	`2.67`
`10 ** 70`	error P2309	an integer of 71 digits
`True + 1`	error P2304	`2`

There is one rounding for all numbers — half away from zero; a `float` is rounded by its decimal representation. Money is `decimal` only: money functions do not accept `float`.

```
Ціна: |(money(2.20 * 3, "UAH")), разом із ПДВ |(money(round(2.20 * 3 * 1.2, 2), "UAH")).
```

10.3. Strings

A string is Unicode characters, as in Python 3: `len("Київ") == 4`. Differences:

- only the methods listed in the specification (section 12.10) exist: `split`, `join`, `strip`, `replace`, `startswith`, `find`, `upper`, `lower`, `title`, `zfill`, `format` and a few others; `center`, `partition`, `encode` are error P2313;
- f-strings follow the Python 3.11 syntax: inside the braces a quote of the same kind as around the string is not allowed (P1208); write `f"{d['k']}"`;
- `"{0.name}".format(r)` is error P2330; a field of a record is read by key: `"{0[name]}"`;
- a format spec (`f"{x:,.2f}"`) is English and knows nothing of the document language; for the text there are `num()`, `money()`, `fmt_date()`.

```
|code:
сума = 1234.5
|end
Python-формат: |(f"{сума:,.2f}"); формат документа: |(num(сума, 2)).
```

10.4. Lists, tuples, dictionaries

As in Python, including list and dict comprehensions and unpacking. Differences:

- there are no sets: `{1, 2}` is error P2101; uniqueness — with a dictionary or `not in`;
- there are no operators `|`, `&`, `^`, `<<`, `~`; dictionaries are merged with `{**a, **b}`;
- `is` only with `None` (P2103);
- a dictionary key is an immutable value; `float` and a length cannot be keys (P2317); `5` and `5.00` are one key;
- `range()` and `reversed()` return a list;
- there is no `lambda`: a function for `key=` is an ordinary `def`.

```
|code:
def мінус(x):
    return -x
порядок = sorted([3, 1, 2], key=мінус)
|end
|(join_list([str(x) for x in порядок]))
```

10.5. Records

A record (`record`) is an immutable value with fields: `r.назва` or `r["назва"]`. Parameters of the YAML header, project data and the results of `load_csv`, `load_yaml`, `load_json` become records. In Python `namedtuple` is similar, but a record is not a sequence:

- `"назва" in r` and `len(r)` are errors; a field is tested with `hasattr(r, "назва")`;
- a missing field is error P2314; `getattr(r, "ціна", 0)` gives a default value;
- a new record with a changed field is `r.replace(кількість=4)`; into a dictionary — `dict(r)`.

```
---
doc: {language: uk-UA}
params:
  сторона: {назва: "ТОВ «Альфа»", код: "12345678"}
---
|(сторона.назва), ІПН: |(getattr(сторона, "ІПН", None) or "—").
```

YAML is read by the rules of Primula: `15.00` is a `decimal` with two digits, a date without quotes is a `datetime`.

10.6. Dates and lengths

A date and a moment are one built-in type `datetime`, without `import`; the precision is a second, there are no time zones.

- `дата + 30`, `дата - 30` shift by days; `дата - дата` is a number of days (`decimal`), not a `timedelta`;
- `hours(36)`, `weeks(2)` are numbers of days: `дата + hours(36)`; calendar months — `add_months(дата, 1)` (31 January + 1 month = 28 February);
- `weekday` is an attribute from 1 (Monday) to 7, not a method from 0;
- a date goes into the text with `fmt_date(дата, "contract")`.

A length is a number with a unit: `10*mm` or `"10mm"`; the constants are `mm`, `cm`, `pt`, `inch` (not `in`). The result is in the unit of the left operand; comparison has a tolerance of 0.001 mm.

```
|code:
підписано = date(2026, 10, 12)
оплата = підписано + 10
|end
Оплата до |(fmt_date(оплата, "contract")) — через |(оплата - підписано) днів, |
(fmt_date(add_months(підписано, 1), "long")) — через місяць.
```

10.7. Functions

`def` with defaults, `*args`, `**kwargs`, keyword-only parameters, recursion, nested functions and closures; inside a function variables are mutable, and there is `while`. Differences:

- there is no `lambda`, no generators (`yield`), no `global`, `nonlocal`; the only decorator is `@export` (section "Modules and fragments");
- default values are computed once and frozen: the Python trap `def f(x, acc=[])` is error P2318 here at the first `append`;
- a global list passed to a function is frozen too: build a new list;
- instead of Python's recursion limit there are limits: depth 500, 20 s for one evaluation, 10 000 000 elements.

```
---
doc: {language: uk-UA}
params:
  позиції:
    - {назва: "Папір", кількість: 20, ціна: 120.50}
    - {назва: "Ручки", кількість: 100, ціна: 15.00}
---
|code:
def рядки(позиції):
    result = []
    for i, p in enumerate(позиції, 1):
        result.append(record(n=i, назва=p.назва, сума=p.кількість * p.ціна))
    return result
|end
|table(cols=["10mm", "*", "35mm"]):
|: № |& Найменування |& Сума, грн
|for(p in рядки(позиції)):
```

```
|: |(p.n) |& |(p.назва) |& |(num(p.сума, 2))
|endfor
|end table
```

10.8. Global names: lazy and immutable

This is the second big difference after numbers. Python runs a module from top to bottom and lets a name be reassigned. In Primula the top-level definitions — `x = …`, `def`, `|let`, header parameters — are collected from the whole file in advance, computed on first use and frozen.

- The order in the text does not matter: a name can be used above its definition.
- A second definition is error P2203, `x += 1` at the top level is P2105, a cycle of definitions is P2204. A header parameter cannot be redefined in code either.
- A definition nobody reads is not computed — and its error does not show until the name is needed.
- The other top-level statements (`for`, `if`, calls) are actions: they run after the definitions are collected, and the names inside them are local (warning P2501).
- A conditional value is a conditional expression: `ставка = 20 if платник_пдв else 0`.
- Accumulation is a list comprehension or a function, not `append` in a top-level loop (P2318).

```
|code:
разом = ціна * кількість
ціна = 250.00
кількість = 4
ставка = 20 if разом > 500 else 0
|end
Разом |(num(разом, 2)), ПДВ |(ставка) %.

|code:
x = 1
x = 2
|end
```

10.9. Modules and fragments

A **module** (`.m.prml`, `|import`) contains only definitions. All its names without a leading `_` are public, nothing needs to be exported; a name with `_` is really private (`модуль._имя` is an error).

```
|code:
_ставка = 20

def з_пдв(сума, ставка=_ставка):
    return round(сума * (100 + ставка) / 100, 2)
|end
```

Import	Access
`|import("lib/ціни")`	`ціни.з_пдв(…)`, a macro — `|ціни.ім'я(…)`
`|import("lib/ціни", "з_пдв")`	`з_пдв(…)` without the prefix
`|import("lib/ціни", alias="ц")`	`ц.з_пдв(…)` (`as` is a Python keyword)

An `|import` of the document also works in the fragments the document includes; an `|import` inside a fragment works only in that fragment. A path is written from the file with the command.

```
|import("lib/ціни")
Разом із ПДВ: |(num(ціни.з_пдв(1000.00), 2)) грн.
```

A **fragment** (`.f.prml`, `|include`) is document text with its own definitions, and they are private: one fragment can be included twice without a conflict. The document sees only what is exported (decision 135):

In the fragment	What is exported
`@export` above a top-level `def f(…)` of `|code:`	the function under its own name
`@export(name="g")` above `def _f(…)`	the function under the name g; `_f` stays private
`|export("підпис", "f")`	names already defined: macros `|def`, functions, `|let`
`|let(x=…, export=True)`, `|export(x=…)`	a new value

```
---
kind: fragment
---
|code:
@export
def знижка(сума, відсоток):
    return round(сума * (100 - відсоток) / 100, 2)
|end
|def рядок(назва, сума):
|(назва) — |(num(сума, 2)) грн
|enddef
|export("рядок")

|include("fragments/знижки")
|рядок("Папір зі знижкою", знижка(2410.00, 5))
```

If two fragments export the same name, the conflict is resolved where they are included: `|include(…, ns="f")` puts the exports into `f.*`, `rename={…}` renames them. `@export` outside a fragment is error P2610, above a nested function P2106, in a module warning P2638 (everything is public there anyway). Functions used by several documents belong in a module; a fragment exports what belongs to its text.

10.10. Values in the text and macros

There is no output: a value reaches the document only through the expression `|(…)`, and the type decides how.

- A string, a number, a date, markup (`content`) are inserted. A fractional number or a date without formatting is warning P2604: format them with `num()`, `money()`, `fmt_date()`.
- `None`, `bool`, a list, a dictionary, a record are error P2603, not "None" or "[1, 2]" as `str()` would give. An optional value — `|(x or "—")`, a list — `join_list()`.
- Markup is opaque: `markup("…") + "…"` is an error; join the strings first, then call `markup()`.
- A function from `|code:` called as a command `|f(…)` inserts its result; a macro `|def` receives the body of the call as the parameter `body`.

```
|(None)

|code:
def жирно(текст):
    return markup("|*{" + текст + "}")
|end
|def позначка(текст, body):
|.Note: |*{|(текст):} |(body)
|enddef
|позначка("Увага"):
Текст із |жирно("виділенням").
|end
```

10.11. What the core does not have

A document must build the same way on any machine, so the core has no input/output (except `load_*`), no clock (`build_date()` is fixed by `--build-date`), no random numbers and no access to Python internals. Forbidden are `import` (modules come in with `|import` in the markup), `class` (data is a `record`), `try`/`raise` (an error stops the build with its position; `fail("…")` is a deliberate error), `lambda`, `yield`, `async`, `global`, `nonlocal`, `del`, `with`, sets, bitwise operators, annotations, `else` on loops, decorators other than `@export`, names like `__class__`. All of this is found when the code is parsed, before evaluation; the full list with replacements is in the example (`forbidden.prml`) and in section 12.2 of the specification.

```
|code:
квадрат = lambda x: x * x
|end
```

11. Libraries: development and use

A Primula library is a directory of `.m.prml` modules and their data used by several projects: checking requisites, names in grammatical cases, payment schedules, a payment QR code. It is developed, tested and released apart from the translator and from the documents: a library is ordinary Primula text, it does not change the language and does not need a build of the translator.

A ready sample is `examples/11-library`: the library `shared/ua-iban` (IBAN UA: checking, groups of four characters, building) and the project `project/` that links it with a symbolic link.

11.1. The libraries of the kit

The repository and the installation package (`share/primula-docs/libraries/`) hold ready libraries of version 0.1.0. Each is a directory with its own `README.md` (functions, sources of the rules, the date they were checked, limits), tests and an example in `demo/`.

Library	What it does	Main functions
`ua_ids`	checks and formats ЄДРПОУ, РНОКПП, IBAN UA	`edrpou`, `rnokpp`, `iban_grouped`, `iban_make`, `party_code`
`names`	full names in cases, initials, signature, positions	`case`, `initials`, `signature`, `position_case`
`ua_data`	NBU and VAT rates, minimum wage, subsistence minimum, CPI, calendar — with the date of effect	`discount_rate_on`, `vat_rate_on`, `min_wage_on`, `cpi_on`, `calendar`
`legal`	penalty, 3 % per annum, inflation losses, terms	`penalty`, `claim_table`, `annual3`, `inflation_losses`, `deadline`
`finance`	annuity, payment schedule, NPV and IRR, straight-line depreciation	`annuity_payment`, `schedule`, `npv`, `irr`, `straight_line`
`estimate`	estimate: totals by sections, discount, VAT, amount in words; commands `|estimate.table(…)`, `|estimate.total_words(…)`	`table`, `total_words`, `totals`, `section_total`
`bib`	bibliography by DSTU 8302:2015: numbered citations `[3, с. 45]` and the list of references; commands `|bib.cite(…)`, `|bib.references(…)`	`registry`, `cite`, `references`, `entry`
`payment_qr`	payment QR by the rules of the NBU (hryvnia)	`payload`, `svg`, `link`
`epc_qr`	SEPA payment QR (euro), EPC069-12	`payload`, `svg`
`codes`	check digits of EAN-13, ISBN, Luhn; EAN-13 bar code	`ean13_check`, `isbn13`, `luhn_check`, `ean13_svg`
`charts`	bars, line and pie for `|figure(svg=…)`	`bar`, `line`, `pie`
`stats`	median, mode, variance, percentile, moving average	`median`, `mode`, `variance`, `percentile`
`electronics`	E3–E96 series, divider, connections, power, tolerance	`series`, `nearest`, `divider`, `parallel`
`vcs`	the change table of a technical document from the Git history	`changes`

A single library is connected by a link `lib` to its directory and imported as `lib/<name>` (as its `README.md` says); with several libraries each has its own link `lib/<name>` and the import `lib/<name>/<name>` (see "Using a library in a project"):

```
ln -s ~/.local/share/primula-docs/libraries/ua_ids lib/ua_ids

|import("lib/ua_ids/ua_ids")
ЄДРПОУ |(ua_ids.edrpou(код)), IBAN |(ua_ids.iban_grouped(рахунок)).
```

What to know:

- A module cannot import a neighbouring library (`../` is forbidden), so `legal` gets the data as an argument: `legal.penalty(борг, початок, сплата, ua_data.series("discount"))`.
- `ua_data` keeps values with a start date and a source. A date after the last record of a series is an error, not a silent "last value": the data must be kept up to date (a new row with a new date; old rows do not change). The data of the library was checked on 31.10.2026, the calendar covers 2024–2026.
- `legal` calculates by the formulas and the data of the library; it is not legal advice. The legal grounds of the calculation and their sources are listed in its `README.md` (the laws were checked against the texts on zakon.rada.gov.ua; the positions of the courts come from the announcements of the Supreme Court and from reviews, not from the full texts of the decisions): check them before using the result in a claim.
- Functions with the suffix `_problem` return the reason as text (an empty string when all is well) and stop nothing; functions without it stop the build on an error.

11.2. What a library can do

A module contains only definitions: functions and constants in `|code:`, macros `|def`, `|import` of other modules (section "Modules: macros and functions"). It can use the whole core — the Python dialect (page "Code: the Python dialect") — and the functions of the standard library: numbers and money, words and cases, dates and workdays, `load_csv`, `load_yaml`, `load_json`, an image from an SVG string (`img(svg=…)`).

What a module does not have, and what to do instead:

Not available	Instead
Python code, `import`, input/output, network, clock	data — files next to the module (`load_yaml("data/…")`), a date — an argument of the function
`doc.*` of the document (a module is evaluated once per build)	the document passes the language and the date as arguments (`lang="uk"`); the functions of the standard library inside a module follow the document language themselves
`ord()`, `chr()`, bitwise operations	the position in an alphabet string: `"ABC…".find(ch)`; tables of values
`assert`	`fail("…")` — stops the build with the message and the place of the call
heavy computations	a loop of the core runs some tens of thousands of steps per second, the limit is 20 s per evaluation; an algorithm of hundreds of thousands of steps (a QR code, a hash) is requested as a function of the translator, and the library builds on it

11.3. Directory layout

```
ua-iban/                 the library (its own repository or directory)
  README.md              purpose, version, licence, functions, sources of the rules
  project.p.prml         the root for the tests: tests/ imports ../iban
  iban.m.prml            the public module: the file name is the namespace (iban.*)
  _tables.m.prml         a helper module (optional)
  data/                  data of the module: YAML, CSV, JSON
  tests/iban-test.prml   test documents
  run-tests.sh           primula check for every test
```

- Names without a leading `_` are the interface of the library; names with `_` are private (`iban._digits` is an error). Change private names freely, public ones only with a new version.
- The first line of a function body is a description string in English, like code comments: `"""IBAN in groups of four."""`. The source shows it, later — the hints of Studio (direction D3).

- The version is the constant `VERSION = "0.1.0"` in the main module and a line in the README; an incompatible change gets a new major number.
- The project file `project.p.prml` is needed by the tests: without it the root is the directory of the test, and `|import("../iban")` leaves the root (P2613). A project that links the library does not read this file.
- Paths of `load_*` and `|import` inside the library are counted from the module file, without `../` outside the library directory.

11.4. Module

```
---
kind: module
---
|code:
VERSION = "0.1.0"
_LETTERS = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"


def normalized(text):
    """IBAN without spaces, in capitals."""
    return "".join(text.split()).upper()


def problem(text):
    """Why the text is not an IBAN UA, or "" if it is valid."""
    s = normalized(text)
    if not s.startswith("UA") or len(s) != 29 or not s[2:].isdigit():
        return "очікується UA і 27 цифр"
    digits = "".join([c if c.isdigit() else str(_LETTERS.find(c) + 10) for c in s[4:]
+ s[:4]])
    return "" if int(digits) % 97 == 1 else "не збігається контрольне число"


def check(text):
    """The normalized IBAN; a wrong one stops the build with the reason."""
    reason = problem(text)
    if reason:
        fail(f"IBAN «{text}»: {reason}")
    return normalized(text)
|end
```

A library reports an error in the data of a document with `fail()`: the build stops with the message of the library, the place of `fail` in the module and the line of the document the call came from. For a soft check a function next to it returns the reason (`problem`) or `True`/`False`.

11.5. Tests

A test is an ordinary document: its code compares the results with the expected ones and calls `fail()` at the first difference. Calls at the top level of `|code:` are actions; they run at every check.

```
|import("lib/iban")
|code:
def expect(name, got, wanted):
    if got != wanted:
        fail(f"{name}: отримано {repr(got)}, очікувалося {repr(wanted)}")


expect("check", iban.check("ua21 3223 1300 0002 6007 2335 6600 1"),
"UA213223130000026007233566001")
expect("problem", iban.problem("UA223223130000026007233566001"), "не збігається
```

```
контрольне число")
|end
Бібліотека ua-iban |(iban.VERSION).
```

`primula check tests/iban-test.prml` exits with code 0 when all expectations match, and with code 2 and the message of `fail` otherwise; `run-tests.sh` of the sample checks all `tests/*.prml`. Test edge cases and errors, not only the happy path: a wrong check digit, a wrong length, another country.

11.6. Using a library in a project

Three ways, from the most convenient for development to the most reproducible:

Way	How	When
symbolic link	`ln -s ../shared/ua-iban lib` in the project root	development, several projects on one machine; the build gives warning P2639
copy	the library directory is copied to `lib/` of the project	release of a document: the project builds the same without outside directories
Git submodule	`git submodule add <url> lib/ua-iban`	a project in Git: the library version is pinned by a commit

```
cd мій-проєкт
ln -s ../shared/ua-iban lib

|import("lib/iban")
IBAN: |(iban.grouped(рахунок)) (код банку |(iban.bank_code(рахунок))).
```

The link is checked on the path as written (decision 136): `lib` inside the project may lead anywhere, while `../` in the text of a document or a module still does not leave the root. The files of the library go to the build report and to `--embed-sources` as `lib/iban.m.prml`, with hashes — the report shows which version of the library built the document.

11.7. Data with a date of effect

Rates, reference lists and calendars change over time, and an old document must build as it did at release. Keep values with the date they take effect and select by the date the document passes (the date of the document, not the build date):

```
vat:
  - {from: 2011-01-01, value: 20}

|code:
_rates = load_yaml("lib/data/rates.yaml")


def vat_on(day):
    """VAT rate on a date; the document passes doc.created or the contract date."""
    found = None
    for r in _rates.vat:
        if r["from"] <= day:
            found = r["value"]
    if found is None:
        fail(f"немає ставки на {fmt_date(day, 'short')}")
    return found
|end
Ставка ПДВ: |(vat_on(date(2026, 10, 26))) %.
```

A new value is added as a row with a new date; old rows do not change.

11.8. What is inside a file: primula describe

`primula describe FILE` shows a summary of a file that is not built into a document.

```
primula describe lib/ua_ids/ua_ids.m.prml        # functions, constants, version of a
module
primula describe styles/проєкт.s.prml            # extends, defaults, defined styles
primula describe project.p.prml --format json    # for editors and Studio
```

For a module it prints the leading `|%` comment, `VERSION`, the public functions with the signature and the first line of the description (a docstring, or the comment above `def`), the public constants and the number of private names; the code is not run. So write a docstring for every function: it goes into the summary. `primula view` shows the same summary for a module, a styles module and a project.

11.9. Release

- README: purpose, functions with examples, version, licence, sources of the rules (standard, law, regulation) — and the date they were checked against.
- Before a release: `run-tests.sh` without errors; the number in `VERSION` and in the README match; the changes are described.
- Do not put real personal data and requisites into the examples of a library — use values that are obviously for training.

For developing a library with AI there is the skill `skills/primula-library`: it follows the same scheme — module, tests, check with the translator.

11.10. Commands of a module: `|def`, `param`, the accumulator

A module can define not only functions but **commands**: a `|def` macro is called as `|module.command(…)` (`|estimate.table(…)`, `|bib.cite(…)`). The name of a macro may coincide with the name of a built-in command (`table`, `list`): the call always has the namespace, the built-in command is not touched.

- The description of a command and its parameters is the `|%` comments right above `|def`; a line `|% name: text` describes a parameter. Studio shows them in the Palette, the hint and the completion, and `primula describe module` and `primula commands --document FILE` give them to other programs.
- The settings of a library come from the parameters of the document with the function `param("name", default)` (for example `currency` of `estimate`, `bib_order` of `bib`): a macro does not see the variables of the document, but it sees the parameters.
- Values that depend on the whole document (the numbers of references by the order of the first reference, a list of terms) are collected by the accumulator: `collect("name", value)` in the text and `collected("name")` after the whole document; if `collected` is called, the document is evaluated twice (section 4.6 of the specification).

Samples are the examples `examples/13-estimate` (an estimate), `14-bibliography` (references by the order of mention) and `15-own-commands` (an own command and a glossary).

12. Importing documents from Word and other formats

A finished contract, manual or act lives in Word, ODT, Markdown or HTML. The command `primula import` turns it into a Primula draft: the text, the headings, the numbered clauses, the lists, the tables, the images, the notes and the links. This is the start of the work, not its end: the draft has to be read and brought to the end — but nothing has to be typed again.

A sample is `examples/16-import`: three sources (`договір.md`, `інструкція.html`, `акт.docx`) and the results of the import next to them.

12.1. The command and the Studio window

```
primula import договір.docx                # договір.prml and assets/ next to it
primula import інструкція.odt -o doc/інструкція.prml --language uk-UA
primula import договір.docx --list-styles   # only list the Word styles, write nothing
```

In Studio — "File → Import a document…" (and "Import into Primula…" in the context menu of a file): the window asks for the path of the result, the language and whether to recognize the structure, and shows the summary.

The source is read by pandoc, so it must be installed (https://pandoc.org); an old binary `.doc` is not read by pandoc — LibreOffice turns it into DOCX first. The Windows build has no pandoc: the import is unavailable there. The format is taken from the extension: `.docx`, `.odt`, `.doc`, `.rtf`, `.md`, `.html`, `.epub`, `.txt`; another reader is `--from`.

An existing result is not replaced without `--force`. After writing, the command runs `primula check` itself and prints the summary: what was converted, what was left out and the result of the check. Exit code 2 — errors remain in the draft (the file is written all the same).

12.2. What is carried over

In the source	In Primula
paragraphs, bold, italic, underline, superscript and subscript, strikeout	paragraphs, `|*{}`, `|/{}`, `|_{}`, `|^{}`, `|,{}`, `|–{}`
headings	`|<`, `|<<`…
nested lists	`|-`, `|#`, `|--`, `|#-`… (paragraphs inside an item — the environment `|list:`)
tables, merged cells, alignment, widths	`|table` with `|&(span=…, rows=…, align=…)`
images	`|img` with sizes in mm, the files go to `assets/` (EMF and WMF of Word — to SVG through LibreOffice)
notes, links, quotes, code	`|footnote{}`, `|link`, `|.Quote:`, `|verbatim`
metadata	`doc.title`, `doc.creator`; the title is also taken from the first line in capitals or from the first heading

Not carried over, and named in the summary: the page layout (headers, footers, margins, columns), direct formatting without a style, tracked changes and comments, Word fields, formulas (their text stays), a table inside a table cell (the text is joined), raw markup.

12.3. How the structure is recognized

In documents from Word the clause numbers are often typed by hand and the headings are just bold lines. The import restores the structure so that Primula makes the numbers and the references afterwards:

- **Typed numbers.** The paragraph "2.1. Покупець оплачує…" becomes a clause `|>`, "2.1.1." a clause `|>>`, a short line "2. Ціна і порядок розрахунків" a heading `|<`. The number is removed from the text: Primula gives the numbering. The typed numbers are compared with the numbering of the result; if they do not match (a "1.2" is missing, a "4." after a "2."), the summary says how many — the numbers of the result follow the structure.
- **Automatic numbering of Word.** A numbered list of one item — this is how Word and ODT store the numbering of headings — becomes a heading (a short text) or a clause (a long sentence); nested lists are the next levels.
- **Bold lines.** A short line entirely in bold after the first heading is a heading of the next level; the lines of the title page before the first heading ("©…", the title) stay text.
- **Table of contents.** A typed table of contents ("Title ….. 5" in a row or a list of links under "Contents") is replaced by `|toc()`.
- **Levels without skips.** The first heading is of the first level, then not deeper than "the previous one + 1"; clauses likewise.

The rules are heuristic: they err where the author of the document followed no rules. That is why the summary names what was done, `--no-structure` switches the recognition off, and `--keep-numbers` leaves the typed numbers in the text (the headings then have no Primula numbers, `number=False`).

12.4. Word styles

A DOCX stores the names of paragraph and character styles, and the import maps them to the styles of Primula:

- the headings of Word ("Heading 2", "Заголовок 2") are headings; `Clause` and `SubClause` are clauses (this is how a DOCX made by `primula export` comes back); "Quote", "Цитата" is `|.Quote:`; ordinary styles ("Normal", "Body Text", "Hyperlink") are an ordinary paragraph; the names of Primula styles (`Body`, `Requisites`, `Strong`…) stay as they are;
- your own styles are described in `import-styles.yaml` — next to the result or in a directory above it (or `--styles FILE`):

```
paragraphs:
  "Пункт": {clause: 1}          # paragraphs of the style "Пункт" are clauses |>
  "Заголовок договору": {heading: 1}
  "Реквізити": {style: Requisites}   # |.Requisites: …
  "Службовий": ignore           # the paragraph is dropped
characters:
  "Важливе": {style: Strong}    # |.Strong{…}
```

The values: `{heading: N}`, `{clause: N}`, `{style: Name}`, `plain` (an ordinary paragraph), `ignore`. A style that is not mapped stays an ordinary paragraph and is named in the summary; `--list-styles` shows all the styles of the source, `--write-styles import-styles.yaml` writes a skeleton for them. For ODT and other formats pandoc gives no style names — only the rules of the structure work there.

12.5. The result

A typical draft is a document without parameters and references:

```
---
doc:
  title: "ДОГОВІР ПОСТАВКИ № 15"
  language: uk-UA
---
ДОГОВІР ПОСТАВКИ № 15

|toc()

|< Предмет договору

|> Постачальник зобов'язується поставити, а Покупець — прийняти й оплатити товар.

|< Ціна і порядок розрахунків

|> Покупець оплачує товар протягом 10 банківських днів.

|>> Зміна ціни можлива лише за згодою сторін.
```

12.6. What to do after the import

1. Run `primula check` (the command already did) and fix the warnings; run `primula build` and look at the result.
2. Replace the references "п. 2.1" in the text with `|@("id")` with an `id=` on the clause (see the page "Contract"): the typed numbers in references do not update themselves.
3. Move the data (parties, dates, amounts) into the parameters of the header or the project and put them in with `|(name)`.
4. Assign styles from the styles of the kit (the page "Styles") instead of `|.Body:` and ordinary paragraphs; signatures and requisites — a table or a style.
5. With many documents — make `import-styles.yaml` from the results of `--list-styles` and import the rest with the same file.
6. To proofread and finish the result — AI in Studio ("Ask AI…" with the recipes "proofreading", "contract terms"): the edits are applied with the check of the translator.

The limits: the import is a draft. It does not guess the meaning (which line is requisites and which is a condition), does not know the layout and does not replace proofreading.

Appendix A. Constructs

Generated by `tools/author_docs.py appendices` from the reference of the translator; do not edit.

Commands

Text

Form	Purpose	Arguments	§
`|link`	external hyperlink; without a body the text is the address	`url`	4.2
`|fmt`	direct formatting of a fragment (a warning: prefer character styles)	`**props`	4.4
`|set`	styles of constructs until the end of the enclosing environment	`**keys`	4.5
`|block`	block style for a group of elements	`style`, `when`	4.3
`|list`	style of one list: a line before the first item, or an environment	`style`	5.4

Structure

Form	Purpose	Arguments	§
`|appendix`	appendix: designation, title, own numbering scope, new page	`id`, `title`, `subtitle`, `number`, `letter`, `page`, `block`, `toc`, `toc_title`, `toc_level`, `when`	5.5
`|scope`	numbering scope with independent counters	`name`, `prefix`	5.5
`|toc`	table of contents (an index updated by Writer)	`max_level`, `title`, `page_break`, `track`	5.9
`|list_of`	list of figures, tables, abbreviations or the alphabetical index	`kind`, `title`, `page_break`, `track`	5.9
`|toc_entry`	table of contents entry pointing to the next paragraph	`text`, `level`	5.9
`|footnote`	footnote at the bottom of the page	`id`	5.10
`|endnote`	endnote	`id`	5.10

Tables

Form	Purpose	Arguments	§
`|table`	table: rows |: … |& …, or rows from data	`cols`, `style`, `width`, `align`, `header`, `footer`, `repeat_header`, `keep_rows`, `may_break`, `frame`, `rules`, `caption`, `id`, `block`, `when`, `data`, `fields`, `titles`, `live`	6.2
`|row`	table row in the expanded form	`align`, `valign`, `padding`, `background`, `paragraph`, `border`, `height`, `min_height`, `keep`, `when`	6.4
`|cell`	table cell in the expanded form	`span`, `rows`, `align`, `valign`, `padding`, `background`, `paragraph`, `border`	6.4
`|caption`	caption of a table with markup		5.8

Pages and graphics

Form	Purpose	Arguments	§
`|page`	change of the page style: from here, or for the environment	`style`, `restart`, `number_format`	7.3
`|newpage`	page break		7.4
`|newcolumn`	column break		7.4
`|section`	section with columns	`cols`, `widths`, `gap`, `rule`, `balance`, `id`, `block`, `when`	7.5

Form	Purpose	Arguments	§
`|header`	page header of a page style	`page`, `which`	7.6
`|footer`	page footer of a page style	`page`, `which`	7.6
`|field.page`	page number (Writer field)		7.6
`|field.pages`	number of pages (Writer field)		7.6
`|field.chapter`	title of the current heading (Writer field)	`level`	7.6
`|field.title`	document title from the metadata (Writer field)		7.6
`|img`	image without a caption	`path`, `key`, `svg`, `place`, `valign`, `x`, `y`, `rel`, `anchor`, `wrap`, `layer`, `gap`, `width`, `height`, `rotate`, `mirror`, `crop`, `opacity`, `border`, `padding`, `style`, `alt`, `when`	8.2
`|figure`	numbered figure; the body is the caption	`path`, `key`, `svg`, `id`, `block`, `caption`, `place`, `valign`, `x`, `y`, `rel`, `anchor`, `wrap`, `layer`, `gap`, `width`, `height`, `rotate`, `mirror`, `crop`, `opacity`, `border`, `padding`, `style`, `alt`, `when`	8.3
`|frame`	text box	`id`, `block`, `place`, `valign`, `x`, `y`, `rel`, `anchor`, `wrap`, `layer`, `gap`, `width`, `height`, `rotate`, `mirror`, `crop`, `opacity`, `border`, `padding`, `style`, `alt`, `when`	8.4
`|watermark`	watermark on all or the listed page styles	`mark`, `pages`, `angle`, `color`, `size`, `opacity`, `font`	8.6
`|parallel`	parallel tracks: texts side by side whose segments start level (bilingual contract, old and new wording, text and comment)	`langs`, `tracks`, `primary`, `widths`, `gap`, `rule`, `header`, `parity`, `style`, `keep_rows`, `id`	7.7
`|segment`	segment of |parallel: a block per track, the starts are level	`id`, `span`, `parity`, `keep`, `status`, `when`	7.7
`|track`	block of a track inside a segment; ends at the next |track or at the end of the segment	`track`, `status`, `rev`, `when`	7.7

Technical documents

Form	Purpose	Arguments	§
`|abbr`	abbreviation: definition and use, or use	`short`, `full`, `text`, `show`	9.4
`|index`	entry of the alphabetical index	`term`, `sub`	9.5

Fragments, modules, parts

Form	Purpose	Arguments	§
`|include`	insert a fragment	`path`, `part`, `key`, `ns`, `rename`, `level_shift`, `**params`	10.2
`|import`	connect a module of definitions	`path`, `*names`, `alias`	10.3
`|document`	part of a compound document (own scope, parameters, new page)	`path`, `scope`, `params`, `page`, `level_shift`, `when`	10.7
`|part`	named part of a fragment for |include(part=…)	`name`	10.2
`|export`	export names of a fragment into the document	`*names`, `**names`	10.4

Code and data

Form	Purpose	Arguments	§
`|let`	define names	`**names`, `export`, `override`	10.4
`|code`	block of code of the computing core		3.7
`|comment`	block comment		3.7

Form	Purpose	Arguments	§
`|verbatim`	literal block	`end`	3.7
`|if`	conditional content	`condition`	9.2
`|elif`	another branch of |if	`condition`	9.2
`|else`	last branch of |if		9.2
`|for`	loop over a list	`loop`	6.9
`|def`	macro: a user command	`signature`	3.2
`|call`	dynamic call of a function as a command	`function`, `*args`	3.2

Short forms

Form	Purpose	§
`||`	literal |	3.3
`|{ |}`	literal braces	3.3
`|;`	empty separator: the end of a command name	3.3
`|%`	comment until the end of the line	3.7
`|(…)`	value of an expression as text	4.2
`|\`	line break in a paragraph	3.3
`|~`	no-break space	3.3
`|*{…}`	bold	3.3
`|/{…}`	italic	3.3
`|_{…}`	underline	3.3
`|^{…}`	superscript	3.3
`|,{…}`	subscript	3.3
`|–{…}`	strike-through	3.3
|{…}`	monospace: designations, codes	3.3
`|"{…}`	quotes of the document language	4.2
`|?{…}`	draft placeholder	4.2
`|.Ім'я{…}`	character style	4.3
`|.Ім'я:`	paragraph style (a line or an environment |end.Ім'я)	4.3
`|@("id")`	cross-reference: number, label, title, page	5.7
`|<`	heading; the number of < is the level (1–6), or level=	5.2
`|>`	clause; the number of > is the level (environment |end>)	5.3
`|-`	bulleted list item; the length is the level	5.4
`|#`	numbered list item; the length is the level	5.4
`|!("kind")`	admonition or note: danger, warning, caution, notice, note, tip	9.1
`|:`	table row	6.4
`|&`	table cell	6.4
`|=(…)`	deferred formula of a cell	6.8

Closing markers

Form	Purpose	§
`|end`	closes the nearest environment, without a check	3.5
`|end name`	closes a named environment, with a check	3.5
`|end>`	closes a clause of this level	3.5
`|end!`	closes an admonition	3.5

Form	Purpose	§
`|end.Ім'я`	closes a paragraph style	3.5
`|endif`	closes |if	3.5
`|endfor`	closes |for	3.5
`|enddef`	closes |def	3.5

Appendix B. Functions of expressions

Generated by `tools/author_docs.py appendices` from the reference of the translator; do not edit.

Numbers

Call	Purpose	Example → result	§
`num(x, digits=None, lang)`	number by the rules of the language: groups, decimal comma	`num(12500, digits=2) → "12,500.00"`	lib. 2
`percent(x, digits=None, lang)`	percent with a no-break space: «20 %»	`percent(20) → "20 %"`	lib. 2
`round_to(x, step)`	round to a step, half away from zero	`round_to(1234, 50) → 1250`	lib. 2
`ceil_to(x, step)`	round up to a step	`ceil_to(1201, 100) → 1300`	lib. 2
`floor_to(x, step)`	round down to a step	`floor_to(1299, 100) → 1200`	lib. 2
`round_money(x)`	round to hundredths, half away from zero	`round_money(dec("10.005")) → 10.01`	lib. 2
`round(x, ndigits=None)`	round, half away from zero; without ndigits — an integer	`round(2.675, 2) → 2.68`	12.4
`abs(x)`	absolute value	`abs(-5) → 5`	12.10
`min(*values, key, default)`	smallest of the arguments or of a list	`min(3, 1, 2) → 1`	12.10
`max(*values, key, default)`	largest of the arguments or of a list	`max([3, 1, 2]) → 3`	12.10
`sum(seq, start=0)`	sum of a list	`sum([1.50, 2.25]) → 3.75`	12.10
`dec(x)`	to decimal (from float — the shortest decimal form)	`dec(0.1) → 0.1`	12.4
`float(x)`	to float; a datetime gives its serial number	`float("2.5") → 2.5`	12.4
`int(x)`	drop the fractional part (a decimal with scale 0)	`int(7.9) → 7`	12.4

Money and VAT

Call	Purpose	Example → result	§
`money(x, code, currency_style="short", lang)`	amount with the currency: «12 500,00 грн»	`money(12500, "UAH") → "12,500.00 UAH"`	lib. 3
`money_words(x, code, kop="digits", cap=False, lang)`	amount in words	`money_words(12500, "UAH") → "twelve thousand five hundred hryvnias 00 kopiykas"`	lib. 3
`money_full(x, code, kop="digits", cap=False, currency_style="short", lang)`	amount in figures and in words in brackets (contract form)	`money_full(12500.50, "UAH") → "12,500.50 UAH (twelve thousand five hundred hryvnias 50 kopiykas)"`	lib. 3
`vat(x, rate)`	tax on an amount without tax	`vat(1000, 20) → 200.00`	lib. 3
`with_vat(x, rate)`	amount with tax	`with_vat(1000, 20) → 1200.00`	lib. 3
`vat_in(x, rate)`	tax contained in an amount with tax	`vat_in(1200, 20) → 200.00`	lib. 3
`without_vat(x, rate)`	amount without tax from an amount with tax	`without_vat(1200, 20) → 1000.00`	lib. 3
`split(total, shares)`	split an amount by shares; the parts add up to the total	`split(100, [1, 1, 1]) → [33.33, 33.33, 33.34]`	lib. 3

In words

Call	Purpose	Example → result	§
`words(x, code=None, gender="m", case="nom", lang)`	number in words; words(x, "UAH") — an amount	`words(21, gender="f") → "twenty-one"`	lib. 4
`ordinal(n, gender="m", case="nom", lang)`	ordinal number in words	`ordinal(21, case="gen") → "twenty-first"`	lib. 4
`date_words(dt, lang)`	date in words	`date_words(date(2026, 10, 12)) → "the twelfth of October, two thousand twenty-six"`	lib. 4

Dates and time

Call	Purpose	Example → result	§
`date(y, m, d)`	date: date(y, m, d) or date("YYYY-MM-DD")	`date(2026, 10, 12) → date("2026-10-12")`	12.3
`datetime(y, m, d, h=0, mi=0, s=0)`	moment: datetime(y, m, d, h, mi, s), an ISO string or a serial number	`datetime("2026-10-12T14:30") → datetime("2026-10-12T14:30:00")`	12.3
`build_date()`	date of the build start (--build-date, build_date: of the project)	`build_date() → date("2026-10-12")`	12.7
`weeks(n)`	number of days in n weeks: dt + weeks(2)	`weeks(2) → 14`	lib. 5
`hours(n)`	number of days in n hours: dt + hours(36)	`hours(36) → 1.5`	lib. 5
`minutes(n)`	number of days in n minutes	`date(2026, 10, 12) + minutes(90) → datetime("2026-10-12T01:30:00")`	lib. 5
`add_months(dt, n)`	shift by months (31 → the last day of the month)	`add_months(date(2026, 1, 31), 1) → date("2026-02-28")`	lib. 5
`add_years(dt, n)`	shift by years	`add_years(date(2028, 2, 29), 1) → date("2029-02-28")`	lib. 5
`fmt_date(dt, format="short", lang)`	date as text: short, long, contract or a template (ДСТУ 4163:2020)	`fmt_date(date(2026, 10, 12), "contract") → "12 October 2026"`	lib. 5
`fmt_time(dt, format="short", lang)`	time as text: short or a template {hh}:{mi}	`fmt_time(datetime(2026, 10, 12, 14, 30, 0)) → "14:30"`	lib. 5
`fmt_datetime(dt, format="short", lang)`	date and time as text	`fmt_datetime(datetime(2026, 10, 12, 14, 30, 0), "long") → "12 October 2026, 14:30"`	lib. 5
`weekday(dt, case="nom", lang)`	name of the day of the week (case nom or acc)	`weekday(date(2026, 10, 12)) → "Monday"`	lib. 5
`month_name(dt, case="nom", lang)`	name of the month (case nom, gen, loc)	`month_name(date(2026, 10, 12), case="gen") → "October"`	lib. 5
`is_workday(dt, calendar)`	whether the day is a working day (by the calendar of the project)	`is_workday(date(2026, 10, 10)) → False`	lib. 5
`workdays(dt, n, calendar)`	the date n working days after dt (n < 0 — before)	`workdays(date(2026, 10, 9), 1) → date("2026-10-12")`	lib. 5

Units and lengths

Call	Purpose	Example → result	§
`qty(x, unit, tol, digits=None, lang)`	value with a unit: «230 В», tol= — tolerance	`qty(230, "V") → "230 V"`	lib. 6
`qty_range(low, high, unit, digits=None, lang)`	range with a unit: «100–240 В»	`qty_range(-20, 55, "degC") → "−20…+55 °C"`	lib. 6
`convert(x, from, to)`	convert between units of one quantity	`convert(1.5, "kW", "W") → 1500`	lib. 6

Call	Purpose	Example → result	§
`length(text)`	length from a string: length("10mm")	`length("10mm") + 5*mm → length("15mm")`	12.3
`mm`	length constant: 10*mm	`10*mm → length("10mm")`	12.3
`cm`	length constant: 10*cm	`10*cm → length("10cm")`	12.3
`pt`	length constant: 10*pt	`10*pt → length("10pt")`	12.3
`inch`	length constant: 10*inch	`10*inch → length("10inch")`	12.3

Text

Call	Purpose	Example → result	§
`plural(n, *forms, frac, lang)`	word form by the number: plural(n, "день", "дні", "днів")	`plural(5, "день", "дні", "днів", lang="uk") → "днів"`	lib. 7
`join_list(items, last, sep=", ", lang)`	join a list: «А, Б та В»	`join_list(["А", "Б", "В"], last="та") → "А, Б та В"`	lib. 7
`cap(s)`	capital first letter	`cap("київ") → "Київ"`	lib. 7
`translit(s, lang)`	transliteration (uk — КМУ № 55, ru — ICAO Doc 9303)	`translit("Згорани", lang="uk") → "Zghorany"`	lib. 7
`sort_key(s, lang)`	sort key by the alphabet of the language: sorted(names, key=sort_key)	`sorted(["Ґава", "Гора", "Єва", "Есе"], key=sort_key) → ['Гора', 'Есе', 'Єва', 'Ґава']`	lib. 7
`nbsp_short(s, lang)`	no-break space after short prepositions and conjunctions	`nbsp_short("в домі та у дворі", lang="uk") → "в домі та у дворі"`	lib. 7
`str(x)`	value as a string (dates — ISO, numbers — with a point)	`str(2.20) → "2.20"`	12.10
`repr(x)`	value as code	`repr("a") → "'a'"`	12.10

Lists and records

Call	Purpose	Example → result	§
`len(seq)`	length: characters, items, keys	`len("Київ") → 4`	12.10
`range(start, stop, step=1)`	numbers from start to stop (not included) with a step	`range(1, 10, 3) → [1, 4, 7]`	12.10
`enumerate(seq, start=0)`	pairs (number, item)	`enumerate(["a", "b"], start=1) → [(1, 'a'), (2, 'b')]`	12.10
`zip(*lists)`	tuples of items with the same index	`zip([1, 2], ["a", "b"]) → [(1, 'a'), (2, 'b')]`	12.10
`sorted(seq, key, reverse=False)`	sorted list	`sorted([3, 1, 2]) → [1, 2, 3]`	12.10
`reversed(seq)`	list in reverse order	`reversed([1, 2, 3]) → [3, 2, 1]`	12.10
`any(seq)`	whether at least one item is true	`any([0, 1]) → True`	12.10
`all(seq)`	whether all items are true	`all([1, 2]) → True`	12.10
`list(seq)`	a list (of a sequence)	`list("abc") → ['a', 'b', 'c']`	12.10
`tuple(seq)`	a tuple	`tuple([1, 2]) → (1, 2)`	12.10
`dict(source=None, **items)`	a dictionary: from pairs, a record or key=value	`dict(a=1, b=2) → {'a': 1, 'b': 2}`	12.10
`record(source=None, **fields)`	a record with access through a point: r.name	`record(name="Кабель", qty=3) → record(name='Кабель', qty=3)`	12.6

Call	Purpose	Example → result	§
`bool(x)`	truth value	`bool("") → False`	12.10
`getattr(r, name, default)`	field of a record by name, with a default	`getattr(record(a=1), "b", 0) → 0`	12.6
`hasattr(r, name)`	whether a record has a field	`hasattr(record(a=1), "a") → True`	12.6
`type(x)`	name of the type: "decimal", "str", "datetime"…	`type(1.5e3) → "float"`	12.3

Data

Call	Purpose	Example → result	§
`group_by(items, key)`	dictionary «key value → list of records»	`group_by([record(k="a", v=1), record(k="a", v=2)], "k") → {'a': [record(k='a', v=1), record(k='a', v=2)]}`	lib. 8
`sum_by(items, key, value)`	sums of a field by groups	`sum_by([record(k="a", v=1), record(k="a", v=2)], "k", "v") → {'a': 3}`	lib. 8
`distinct(items, key)`	unique values of a field	`distinct([record(k="a"), record(k="b"), record(k="a")], "k") → ['a', 'b']`	lib. 8
`load_csv(path, types, sep)`	rows of a CSV file as records	`load_csv("data/items.csv")`	10.5
`load_yaml(path)`	data of a YAML file (records and lists)	`load_yaml("data/order.yaml")`	10.5
`load_json(path)`	data of a JSON file	`load_json("data/order.json")`	10.5
`load_text(path)`	the whole text file as a string (UTF-8; e.g. SVG for |img(svg=…))	`load_text("img/sign.svg")`	10.5
`load_ods(path, sheet, range, header=True, types)`	rows of a sheet of an ODS file as records (saved results of formulas)	`load_ods("data/items.ods", sheet="Позиції")`	10.5
`load_xlsx(path, sheet, range, header=True, types)`	rows of a sheet of an XLSX file as records (saved results of formulas)	`load_xlsx("data/items.xlsx", sheet="Позиції")`	10.5

Mathematics (std/math)

Call	Purpose	Example → result	§
`math.sqrt(x)`	square root; in a document: |import("std/math")	`math.sqrt(2) → 1.4142135623730951`	lib. 9
`math.exp(x)`	e to the power x; in a document: |import("std/math")	`math.exp(1) → 2.718281828459045`	lib. 9
`math.log(x)`	natural logarithm; in a document: |import("std/math")	`math.log(math.e) → 1.0`	lib. 9
`math.log10(x)`	decimal logarithm; in a document: |import("std/math")	`math.log10(1000) → 3.0`	lib. 9
`math.sin(x)`	sine (radians); in a document: |import("std/math")	`math.sin(math.pi / 2) → 1.0`	lib. 9
`math.cos(x)`	cosine (radians); in a document: |import("std/math")	`math.cos(0) → 1.0`	lib. 9
`math.tan(x)`	tangent (radians); in a document: |import("std/math")	`math.tan(0) → 0.0`	lib. 9
`math.atan2(y, x)`	angle of the point (x, y), radians; in a document: |import("std/math")	`math.atan2(1, 1) → 0.7853981633974483`	lib. 9
`math.floor(x)`	round down to an integer (decimal); in a document: |import("std/math")	`math.floor(-2.5) → -3`	lib. 9

Call	Purpose	Example → result	§
`math.ceil(x)`	round up to an integer (decimal); in a document: | import("std/math")	`math.ceil(2.1) → 3`	lib. 9
`math.pi`	the number π (float); in a document: |import("std/math")	`math.pi → 3.141592653589793`	lib. 9
`math.e`	the number e (float); in a document: |import("std/math")	`math.e → 2.718281828459045`	lib. 9

QR codes and encodings

Call	Purpose	Example → result	§
`qr_matrix(data, ecc="M", min_version=1, max_version=40, encoding="utf-8")`	QR code of a text (byte mode): {version, ecc, mask, size, rows}	`qr_matrix("https://primula.example").version → 2`	lib. 10
`qr_svg(data, ecc="M", min_version=1, max_version=40, encoding="utf-8", size="30mm", quiet=4, circle=0)`	QR code as SVG for |img(svg=…); circle= leaves a white circle in the centre	`len(qr_svg("https://primula.example")) > 0 → True`	lib. 10
`base64url(text, encoding="utf-8")`	Base64URL of the bytes of a text, without = padding	`base64url("Київ") → "0JrQuNGX0LI"`	lib. 10

Checks and document

Call	Purpose	Example → result	§
`fail(*message)`	stop the build with an error at this place	`fail("no price")`	12.9
`debug(*values)`	write values into the build report, not into the document	`debug(total)`	12.9
`has(feature)`	whether the product variant has a feature (keys: of the project, --feature)	`has("wifi")`	9.2
`markup(text)`	Primula markup from a string (content)	`markup("|*{" + name + "}")`	4.2
`param(name, default)`	a parameter of the document by name (also inside a module: its settings), or the default (decision 144)	`param("currency", "UAH")`	10.6
`collect(name, value)`	add a value to a collection of the document; the result is empty content (decision 143)	`|(collect("cited", key))`	4.6
`collected(name)`	the values of a collection after the whole document, in the order of collect(); evaluates the document twice, [] in the first pass (decision 143)	`collected("cited")`	4.6
`img(path, svg, **options)`	image value for watermark=, |img(value), |figure(value); a path or svg=	`img("assets/stamp.png")`	8.6
`col(width, id, type, **props)`	table column: col(width, id=…, type=…, align=…)	`col("20mm..35mm", id="sum", type="money", align="right")`	6.3
`track(id, lang, title, **props)`	track of |parallel: track(id, lang=…, title=…, width=…, primary=…)	`track("note", title="Коментар", width="28%", numbered=False, optional=True)`	7.7

Table formulas

Call	Purpose	Example → result	§
`this_row(c)`	value in column c of the current row	`this_row("qty") * this_row("price")`	6.8
`cell(r, c)`	value of a cell	`cell(2, "sum")`	6.8
`column(c, rows)`	list of the values of a column (body rows or rows=)	`sum(column("sum"))`	6.8

Call	Purpose	Example → result	§
`body_rows()`	numbers of the rows between the header and the totals	`body_rows()`	6.8
`avg(seq)`	average of a list	`avg(column("qty"))`	6.8
`count(seq)`	number of non-empty values	`count(column("qty"))`	6.8

Styles code

Call	Purpose	Example → result	§
`paragraph_style(name, parent, **props)`	define a paragraph style: name, parent=…, properties (11.3)	`paragraph_style("MyStyle", parent="…")`	11.2
`character_style(name, parent, **props)`	define a character style: name, parent=…, properties (11.3)	`character_style("MyStyle", parent="…")`	11.2
`block_style(name, parent, **props)`	define a block style: name, parent=…, properties (11.3)	`block_style("MyStyle", parent="…")`	11.2
`table_style(name, parent, **props)`	define a table style: name, parent=…, properties (11.3)	`table_style("MyStyle", parent="…")`	11.2
`list_style(name, parent, **props)`	define a list style: name, parent=…, properties (11.3)	`list_style("MyStyle", parent="…")`	11.2
`frame_style(name, parent, **props)`	define a frame style: name, parent=…, properties (11.3)	`frame_style("MyStyle", parent="…")`	11.2
`page_style(name, parent, **props)`	define a page style: name, parent=…, properties (11.3)	`page_style("MyStyle", parent="…")`	11.2
`derive(base, name, **props)`	derived style: its deterministic name for |.(…)	`derive("Clause", left="10mm")`	11.4
`structure_styles()`	styles of structure constructs: paragraph=, heading=, clause=…	`structure_styles(paragraph="Body")`	11.5
`structure_profile()`	structure profile: "business" or "report"	`structure_profile("business")`	5.1
`numbering(headings, heading_format, clause_format, figures, tables, ref_format)`	numbering of headings, clauses, figures and tables	`numbering(figures="by_chapter")`	5.2
`captions(lang, figure, table)`	caption words of figures and tables	`captions("uk", figure="Рисунок", table="Таблиця")`	5.8
`appendices(lang, designation, numbering, letters, page_break, toc, title_format)`	designation and layout of appendices	`appendices("uk", designation="ДОДАТОК")`	5.9
`ref_labels(lang)`	words of references |@(id, "label")	`ref_labels("uk", clause="п.")`	5.7
`footnotes(numbering, restart, format, separator)`	numbering and separator of footnotes	`footnotes(restart="page")`	5.10
`abbreviations(first, sort)`	first use and sorting of abbreviations	`abbreviations(first="expand")`	9.4
`typography(percent_space, nbsp_short, nbsp_words)`	typography: percent_space, nbsp_short, nbsp_words	`typography(nbsp_short=True)`	lib. 7
`admonition_symbols(danger, warning, caution, notice, note, tip)`	safety sign images of admonitions	`admonition_symbols(danger="assets/signs/w012.svg")`	9.1
`admonition_words(lang, danger, warning, caution, notice, note, tip, notes)`	signal words of admonitions	`admonition_words("uk", danger="НЕБЕЗПЕКА")`	9.1
`review_marks(lang, true, false, watermark)`	texts of the --review marks and watermark	`review_marks("uk", watermark="НЕ ДЛЯ ВИПУСКУ")`	9.2
`watermark()`	watermark on the page styles	`watermark("ПРОЄКТ")`	8.6
`header()`	header of a page style	`header("Portrait", markup("…"))`	7.6

Call	Purpose	Example → result	§
`footer()`	footer of a page style	`footer("Portrait", markup("…"))`	7.6
`placeholders()`	|?{…} placeholders: "mark" (draft) or "final"	`placeholders("final")`	4.2
`quotes()`	quotation marks of a language for |"{…}	`quotes("de", ("„", "“"), (",", "‘"))`	4.2
`currency(code, lang, short, units, frac, gender, minor, minor_gender, minor_digits)`	add or change a currency of the library	`currency("PLN", "uk", short="zł")`	lib. 1
`unit(unit, lang, symbol, quantity, factor, offset)`	add or change a unit of the library	`unit("rpm", "uk", symbol="об/хв", quantity="frequency", factor=1/60)`	lib. 1
`month_names(lang, nom, gen, loc)`	names of the months of a language	`month_names("uk", gen=["січня", …])`	lib. 1

Project

Call	Purpose	Example → result	§
`build(source, output, params, keys, pdf)`	register a build target of the project	`build("act.prml", params=row, output=f"out/act-{row.no}.odt")`	10.7
`batch_mode()`	True while the whole project is built	`batch_mode()`	10.6

Methods

Call	Purpose	Example → result	§
`str.split(sep=None, maxsplit=-1)`	split by a separator (by spaces without it)	`"a, b".split(", ") → ['a', 'b']`	12.10
`str.rsplit(sep=None, maxsplit=-1)`	split from the right	`"a.b.c".rsplit(".", 1) → ['a.b', 'c']`	12.10
`str.splitlines(keepends=False)`	list of lines	`"a\nb".splitlines() → ['a', 'b']`	12.10
`str.join(items)`	join strings with this string between them	`", ".join(["a", "b"]) → "a, b"`	12.10
`str.strip(chars=None)`	without spaces (or the characters) at both ends	`"  a ".strip() → "a"`	12.10
`str.lstrip(chars=None)`	without spaces at the start	`"  a".lstrip() → "a"`	12.10
`str.rstrip(chars=None)`	without spaces at the end	`"a  ".rstrip() → "a"`	12.10
`str.replace(old, new, count=-1)`	replace a substring	`"a-b".replace("-", "/") → "a/b"`	12.10
`str.startswith(prefix)`	whether it starts with a prefix	`"ДСТУ 3008".startswith("ДСТУ") → True`	12.10
`str.endswith(suffix)`	whether it ends with a suffix	`"act.prml".endswith(".prml") → True`	12.10
`str.find(sub, start=None, end=None)`	index of a substring or −1	`"abc".find("c") → 2`	12.10
`str.rfind(sub, start=None, end=None)`	index of the last occurrence or −1	`"abca".rfind("a") → 3`	12.10
`str.count(sub, start=None, end=None)`	number of occurrences	`"banana".count("a") → 3`	12.10
`str.upper()`	upper case	`"київ".upper() → "КИЇВ"`	12.10
`str.lower()`	lower case	`"КИЇВ".lower() → "київ"`	12.10

Call	Purpose	Example → result	§
`str.capitalize()`	first letter capital, the rest small	`"кИЇВ".capitalize() → "Київ"`	12.10
`str.title()`	each word with a capital letter	`"нова пошта".title() → "Нова Пошта"`	12.10
`str.isdigit()`	whether only digits	`"2026".isdigit() → True`	12.10
`str.isalpha()`	whether only letters	`"Київ".isalpha() → True`	12.10
`str.isspace()`	whether only spaces	`" ".isspace() → True`	12.10
`str.zfill(width)`	pad with zeros on the left	`"7".zfill(3) → "007"`	12.10
`str.format(*args, **kwargs)`	substitute values: "{0} — {name}"	`"{0} шт.".format(3) → "3 шт."`	12.10
`list.append(x)`	add an item (inside functions)	`[1].append(2)`	12.10
`list.extend(seq)`	add the items of a list	`[1].extend([2, 3])`	12.10
`list.insert(i, x)`	insert an item at an index	`[1].insert(0, 0)`	12.10
`list.pop(i=-1)`	remove and return an item (the last one)	`[1, 2].pop() → 2`	12.10
`list.remove(x)`	remove the first equal item	`[1, 2].remove(1)`	12.10
`list.index(x)`	index of an item	`["a", "b"].index("b") → 1`	12.10
`list.count(x)`	number of equal items	`[1, 1, 2].count(1) → 2`	12.10
`list.sort(key, reverse=False)`	sort in place	`[2, 1].sort()`	12.10
`list.reverse()`	reverse in place	`[1, 2].reverse()`	12.10
`dict.get(key, default=None)`	value by a key or a default	`{"a": 1}.get("b", 0) → 0`	12.10
`dict.keys()`	list of the keys	`{"a": 1}.keys() → ['a']`	12.10
`dict.values()`	list of the values	`{"a": 1}.values() → [1]`	12.10
`dict.items()`	pairs (key, value)	`{"a": 1}.items() → [('a', 1)]`	12.10
`dict.update(other)`	add the pairs of another dictionary	`{"a": 1}.update({"b": 2})`	12.10
`dict.pop(key, default)`	remove a key and return its value	`{"a": 1}.pop("a") → 1`	12.10
`dict.setdefault(key, default=None)`	value by a key; absent — set the default	`{}.setdefault("a", 1) → 1`	12.10

Appendix C. YAML header keys

Generated by `tools/author_docs.py appendices` from the reference of the translator; do not edit.

Key	Files	Type	Purpose	Example → result	§
`kind`	document, fragment, module, styles, project	string	file kind; implied by the sub-extension (.f, .m, .s, .p), required in a project	`kind: document`	2.2
`primula`	document, fragment, module, styles, project	mapping	language declaration: {version: "1.1"}, optionally the library version	`primula: {version: "1.1"}`	2.3
`primula.version`	document, fragment, module, styles, project	string	language version; only 1.1 is accepted	`version: "1.1"`	2.3
`primula.stdlib`	document, fragment, module, styles, project	string	minimal version of the standard library	`stdlib: "1.1"`	13
`doc`	document	mapping	ODT metadata of the document; in the text | (doc.title)	`doc:`	2.3
`doc_defaults`	project	mapping	defaults of the metadata (doc:) of every document of the project; a document overrides them key by key	`doc_defaults:`	10.6
`doc.title`	document	string	title (dc:title) (recommended)	`title: "Договір поставки"`	2.3
`doc_defaults.title`	project	string	title (dc:title)	`title: "Договір поставки"`	10.6
`doc.subject`	document	string	subject (dc:subject)	`subject: "Поставка обладнання"`	2.3
`doc_defaults.subject`	project	string	subject (dc:subject)	`subject: "Поставка обладнання"`	10.6
`doc.description`	document	string	description (dc:description)	`description: "Типова форма"`	2.3
`doc_defaults.description`	project	string	description (dc:description)	`description: "Типова форма"`	10.6
`doc.creator`	document	string	author (meta:initial-creator, dc:creator) (recommended)	`creator: "Іван Петренко"`	2.3
`doc_defaults.creator`	project	string	author (meta:initial-creator, dc:creator) (recommended)	`creator: "Іван Петренко"`	10.6
`doc.organization`	document	string	organization (user-defined property)	`organization: "ТОВ «Постачальник»"`	2.3

Key	Files	Type	Purpose	Example → result	§
`doc_defaults.organization`	project	string	organization (user-defined property) (recommended)	`organization: "ТОВ «Постачальник»"`	10.6
`doc.language`	document	language	document language (dc:language); else the project language (recommended)	`language: uk-UA`	2.3
`doc_defaults.language`	project	language	document language (dc:language); else the project language	`language: uk-UA`	10.6
`doc.identifier`	document	string	document number (user-defined property)	`identifier: "Д-2026-017"`	2.3
`doc_defaults.identifier`	project	string	document number (user-defined property)	`identifier: "Д-2026-017"`	10.6
`doc.revision`	document	string	revision (user-defined property)	`revision: "2"`	2.3
`doc_defaults.revision`	project	string	revision (user-defined property)	`revision: "2"`	10.6
`doc.status`	document	string	status (user-defined property)	`status: "проєкт"`	2.3
`doc_defaults.status`	project	string	status (user-defined property)	`status: "проєкт"`	10.6
`doc.created`	document	date	creation date (meta:creation-date); else the build date (recommended)	`created: 2026-10-12`	2.3
`doc_defaults.created`	project	date	creation date (meta:creation-date); else the build date	`created: 2026-10-12`	10.6
`doc.modified`	document	date	modification date (dc:date); else the build date	`modified: 2026-10-16`	2.3
`doc_defaults.modified`	project	date	modification date (dc:date); else the build date	`modified: 2026-10-16`	10.6
`doc.keywords`	document	strings	keywords (meta:keyword): a string or a list	`keywords: [договір, поставка]`	2.3
`doc_defaults.keywords`	project	strings	keywords (meta:keyword): a string or a list	`keywords: [договір, поставка]`	10.6
`params`	document, fragment, project	mapping	parameters with default values; a target or --param overrides them	`params: {vat: true, delivery_days: 5}`	10.6
`structure`	document	mapping	structure profile settings of the document	`structure: {numbering: {headings: false}}`	5.1
`styles`	document, project	paths	styles modules: a list of paths or one path (std/styles/default — the built-in one)	`styles: [styles/business.s.prml]`	11.7
`references`	document	list	documents of the project whose targets the document refers to (names from ref_documents of the project)	`references: [contract]`	5.7
`defaults`	styles	mapping	default properties: font, size, language	`defaults: {font: Roboto, size: 11pt, language: uk-UA}`	11.1
`paragraphs`	styles	mapping	paragraph styles	`paragraphs:`	11.2
`characters`	styles	mapping	character styles	`characters:`	11.2
`blocks`	styles	mapping	block styles	`blocks:`	11.2
`tables`	styles	mapping	table styles	`tables:`	11.2

Key	Files	Type	Purpose	Example → result	§
`lists`	styles	mapping	list styles	`lists:`	11.2
`frames`	styles	mapping	frame styles	`frames:`	11.2
`extends`	styles	path	base styles module, loaded as the layer before this one (decision 133)	`extends: std/styles/business`	11.7
`pages`	styles	mapping	page styles	`pages:`	11.2
`name`	project	string	project name	`name: "Договори 2026"`	10.6
`language`	project	language	default language of the documents	`language: uk-UA`	10.6
`extends`	project	path	base project: its parameters, keys, reference documents, styles and metadata defaults are inherited	`extends: ../../project.p.prml`	10.6
`on_conflict`	project	string	export conflicts of fragments: error, first, last	`on_conflict: error`	10.4
`keys`	project	mapping	key map: name → path	`keys: {logo: assets/logo.svg}`	9.3
`limits`	project	mapping	core limits: time, recursion, size	`limits: {time: 20, recursion: 500, size: 10_000_000}`	12.8
`limits.time`	project	number	time limit, seconds	`time: 20`	12.8
`limits.recursion`	project	int	recursion depth limit	`recursion: 500`	12.8
`limits.size`	project	int	size limit of values	`size: 10_000_000`	12.8
`build_date`	project	date	fixed build date (today() and the default dates)	`build_date: 2026-10-12`	12.7
`targets`	project	list	build targets of primula build project.p.prml	`targets:`	10.7
`targets[].source`	project	path	source document	`- source: contract.prml`	10.7
`targets[].output`	project	path	output ODT	`output: out/contract.odt`	10.7
`targets[].params`	project	mapping	parameter values of the target	`params: {vat: false}`	10.7
`targets[].keys`	project	mapping	keys of the target	`keys: {logo: assets/other.svg}`	10.7
`targets[].pdf`	project	any	also a PDF: true or its path	`pdf: true`	10.7
`ref_documents`	project	mapping	documents referred to by others (references: of a document): name: path or {source, registry}	`ref_documents: {contract: contract.prml}`	10.6

Appendix D. Messages of the translator

Generated by `tools/author_docs.py appendices` from the reference of the translator; do not edit.

Code	Kind	Message	§
P0001	error	'{command}' is not implemented at prototype stage {stage}; no output is written	— (временный, для команд прототипа)
P0002	error	internal error of the translator: {detail}	14.2
P0004	error	the time limit of the build (--timeout) is exceeded	12.8, 14.1 (`--timeout`)
P0005	error	out of memory: the evaluation exceeded the memory limit of the translator process	12.8
P0006	error	interrupted: the evaluation of the input was stopped	14.1 (`primula console`, решение 128)
P0101	error	cannot read file '{path}': {reason}	14.1
P0102	error	the file is not valid UTF-8: invalid byte at offset {byte_offset}	2.1
P0103	error	cannot write file '{path}': {reason}	14.3
P0105	error	--overlay '{text}' must be written as FILE=BUFFER, and the buffer file must exist	14.1, решение 124
P0106	error	primula parallel: {problem}{value}	7.7, 14.1, решение 134
P0107	error	segment {segment}, track '{track}' is not in the state '{expected}': {reason} ({status})	7.7, 14.1, решение 134
P1101	error	the YAML header is not closed: a line '---' is missing	2.3, А `Header`
P1102	error	invalid YAML in the header: {problem}	2.3
P1103	error	the YAML header must be a mapping of keys to values, found {found}	2.3
P1104	warning	unknown key '{key}' in the header of a file of kind '{kind}'	2.3, 10.6, 11.1
P1105	error	header key '{key}' must be {expected}, found {found}	2.3, 10.6, 11.1
P1106	error	unknown file kind '{value}'; expected one of: {expected}	2.2
P1107	error	unsupported language version '{value}'; this translator implements version {supported}	0.1, 2.3
P1109	error	non-existent date '{value}'	12.6 (решение 25)
P1110	error	duplicate key '{key}'	YAML 1.2 (уникальность ключей)
P1111	error	unsupported YAML tag '{tag}'	12.6 (схема core)
P1112	error	recursive YAML alias: a value contains itself	12.6
P1113	error	a mapping key must be a scalar value	12.6
P1114	warning	'{value}' is a string in YAML 1.2, not a boolean	12.6 (решение 25)
P1115	warning	'{value}' is read as the integer {number}: the leading zeros are lost	12.6 (решение 25)
P1116	error	the key 'lines' is removed: the line mode was removed in version 1.1, a line form ends at the end of the source line	2.3, 3.4, решение 107
P1117	error	invalid standard library version '{value}': expected MAJOR.MINOR, for example "1.1"	2.3, 13

Code	Kind	Message	§
P1118	error	the file requires standard library {value}; this translator has {available}	2.3, 13
P1119	error	time zones are not supported: '{value}'	2.3, 12.6, решение 109
P1120	error	header key '{key}' was renamed: write '{new}:' (decision 129)	2.3, 10.6, решение 129
P1201	error	'|' must be followed by a command name, '(' or a short form; found {found}	3.1
P1202	error	unknown short form '|{char}'	3.1, B.1
P1203	error	'|{char}' is not a short form	3.1
P1204	error	'{bracket}' is not closed	3.2, решение 22
P1205	error	closing '{found}' does not match the opening bracket; expected '{expected}'	3.2
P1206	error	the string literal is not closed	3.2
P1207	error	invalid Python syntax: {problem}	12.2
P1208	error	an f-string contains nested quotes of its own kind; Primula uses the Python 3.11 syntax	0.2, решение 30
P1209	error	'|{form}' needs a body '{{…}}'	3.3, A `InlineShort`
P1210	error	'|{form}' is allowed only at the start of a line	3.3, B.1
P1211	error	a letter directly follows the command name '{name}'	3.2, A `InlineCommand`
P1212	error	'|{form}' must be followed by a space and text	A `Heading`, `Clause`, `ListItem`
P1213	error	heading level {level} is too deep; levels 1–6 are allowed	5.2
P1214	error	'|!' needs arguments with the kind of the admonition, for example |!("warning")	9.1, A `Admonition`
P1215	error	'{{' is not closed	3.2, A `InlineBody`
P1216	error	'{name}' is not closed; expected '{close}'	3.5
P1217	error	'{found}' does not close '{name}'; expected '{expected}'	3.5
P1218	error	'{found}' has no open environment to close	3.5
P1219	error	'|end' cannot close '|{kind}'; use '|end{kind}'	3.5, решение 28
P1220	error	'|{form}' without a matching '|if'	3.6, решение 21
P1221	error	a closing marker must occupy a whole line	3.5, решение 34
P1222	error	the line has a smaller or different indentation than the line opening the block	3.7, решение 24
P1223	error	the verbatim block is not closed: no line '{marker}'	3.7, решение 34
P1224	error	the code block is not closed: no line '|end'	2.2, 3.7
P1225	error	'|verbatim' accepts only the argument end="…" with a non-empty one-line string	3.7, A `VerbatimBlock`
P1226	error	'{what}' is not allowed directly inside '|{container}'	6.2, A 3.5
P1227	error	'|&' is allowed only in a table row	6.4
P1228	error	a formula '|=(…)' is allowed only in a table cell	6.8
P1229	error	invalid macro header; expected '|def name(parameters):'	A `DefBlock`
P1230	error	a heading is not allowed inside a clause environment	5.3
P1231	error	'{form}' is not allowed inside a list environment	5.4, решение 33
P1232	error	list level {level} follows level {previous}: a level is skipped	5.4, решение 32
P1233	error	list marker '{marker}' does not match the parent items; expected '{expected}'	5.4, решение 32

Code	Kind	Message	§
P1234	error	bullet and numbered items are mixed on level {level} of one list	5.4, решение 32
P1235	error	'{name}' is a reserved name and cannot be defined	B.3
P1236	warning	this line starts a new paragraph after a line form	3.4, 14.2
P1238	warning	an environment of {lines} lines is closed with an anonymous '|end'; write '{close}'	3.5, 14.2, решение 35
P1239	error	'|list(…)' is not followed by a list	5.4
P1240	error	'|{form}' after '|else'	3.6, A IfBlock
P1241	error	a list environment must start with a list item	5.4, решение 33
P1242	error	expected {expected} after '{after}'	A
P1243	error	a table row '|:' is allowed only inside '|table'	6.4, A 3.5
P1244	error	'level=' cannot be combined with {count} '<': give the level either by the number of '<' or by level= with a single '<'	3.3, 5.2, решения 105, 114
P1245	warning	'|@(…)' has no body: the text in braces after it is printed as it is	5.7, решение 141
P2101	error	'{construct}' is not allowed in Primula code	12.2
P2102	error	the name '{name}' is not allowed: names with leading underscores reach Python internals	12.2, 12.7
P2103	error	'is' and 'is not' are allowed only with None	12.2
P2104	error	'{statement}' outside a function or loop	12.2
P2105	error	'{name} …=' at the top level redefines a global name; global names are immutable	12.7, решение 40
P2106	error	'@export' is allowed only as '@export' or '@export(name="…")' on a def at the top level of a code block	10.4, 12.2, решение 135
P2201	error	name '{name}' is not defined	10.4
P2202	error	local name '{name}' is read before it is assigned	12.7
P2203	error	'{name}' is already defined	10.4
P2204	error	cyclic definition: {chain}	10.4
P2301	error	operation '{op}' is not supported for '{left}' and '{right}'	12.3
P2303	error	division by zero	12.4
P2304	error	bool takes no part in arithmetic; convert with int(…)	12.4, решение 41
P2305	error	'{op}' is not supported between '{left}' and '{right}'	12.3, решение 43
P2307	error	cannot convert {value} to {type}	12.4, 12.10
P2308	error	invalid numeric operation ({detail})	12.4
P2309	error	number overflow: the result has more than {limit} significant digits	12.4, решение 108
P2310	error	time zones are not supported: {value}	12.3, решение 109
P2311	error	index {index} is out of range (size {size})	12.10
P2312	error	key {key} not found	12.10
P2313	error	'{type}' has no attribute '{name}'	12.10, решение 43
P2314	error	the record has no field '{name}'	12.6
P2315	error	'{type}' is not iterable	12.10
P2316	error	'{type}' does not support '{operation}'	12.3
P2317	error	'{type}' cannot be a dictionary key	12.3, решение 43
P2318	error	a frozen {type} cannot be changed	12.7, решения 40, 46
P2319	error	a {type} cannot be changed while it is being iterated	12.7

Code	Kind	Message	§
P2320	error	`cannot unpack: expected {expected} values, got {got}`	12.2
P2321	error	`{function}(): {problem}`	12.2
P2322	error	`{function}: {problem}`	12.10
P2324	error	`cannot assign to this target`	12.2
P2325	error	`the field '{name}' is given twice (names are compared after NFKC normalisation)`	3.9, 12.6, решение 102
P2330	error	`attribute access in a format field is not allowed: '{{{field}}}'; use an index, for example {{0[name]}}`	12.10, решение 45
P2331	error	`invalid format: {problem}`	12.10
P2340	error	`fail: {message}`	12.9
P2402	error	`the recursion limit ({limit}) is exceeded`	12.8
P2403	error	`a collection of {size} elements exceeds the size limit ({limit})`	12.8
P2404	error	`a string of {size} characters exceeds the size limit ({limit})`	12.8
P2406	error	`the time limit of one evaluation ({limit} s) is exceeded`	12.8
P2501	warning	`'{name}' is assigned in a top-level statement but never used there; it does not become a global name`	12.7, решение 40
P2502	warning	`the name '{name}' mixes alphabets ({scripts})`	3.9, 14.2, решение 102
P2503	warning	`the name '{name}' differs from '{other}' only by look-alike letters`	3.9, 14.2, решение 102
P2601	error	`unknown command '|{name}': it is neither a built-in command nor a defined macro or function`	3.2
P2602	error	`'{name}' is a value of type {type}, not a command`	3.2
P2603	error	`a value of type {type} cannot be inserted into text`	4.2, решение 47
P2604	warning	`a value of type {type} is inserted into text without formatting`	4.2, 14.2, решение 47
P2605	error	`the content of '{name}' has several blocks and cannot be inserted inside a line`	3.2, решение 48
P2606	error	`macro '{name}' has no 'body' parameter, but the call passes a body`	3.2, решение 48
P2608	error	`'{what}' in a branch of a top-level |if: a definition cannot be conditional`	10.4, решение 49
P2609	error	`|{command}: {problem}`	10.4
P2610	error	`only a fragment included outside |if, |for, |def can export names`	10.4, решение 49
P2611	error	`when= must be a boolean, found {type}`	9.2, решение 29
P2612	error	`the value must be known when the tree is built: '{name}' is not a header parameter`	10.2, решение 50
P2613	error	`invalid path '{path}': URLs, absolute paths and paths outside the project root are not allowed`	10.1
P2614	error	`cannot read '{path}': {reason}`	10.1
P2615	error	`cycle of inclusion: {chain}`	10.1; 11.7 — цикл основ `extends` модулей стилей (решение 133)
P2616	error	`'{path}' is a file of kind '{kind}', expected '{expected}'`	10.2, 10.3; 11.7 — основа `extends` не модуль стилей
P2617	error	`unknown parameter '{name}' of the fragment '{path}'`	10.2
P2618	error	`part '{part}' not found in '{path}'`	10.2

Code	Kind	Message	§
P2619	error	unknown key '{key}': it is not in the key map of the project	9.3
P2620	error	'{name}' is exported by two fragments; separate them with ns= or rename=	10.4
P2621	error	'{name}' is defined in the document and exported by a fragment; use override=True or rename=	10.4
P2622	error	the module name '{name}' is not an identifier or is reserved	10.3, решение 52
P2623	error	only definitions are allowed in a module: |def, |let, |code:, |import	10.3, решение 52
P2624	error	'{name}' is not a public name of the module '{module}'	10.3, решение 52
P2625	error	standard library module '{name}' is not available at this prototype stage	13.1
P2626	error	the parameter 'features' must be a list of strings	9.2, решение 53
P2629	error	'{value}' in column '{column}' is not a valid {type}	10.5, решение 51
P2630	error	invalid data: {problem}	10.5, решения 25, 51
P2632	error	|call needs a function as its first argument	3.2
P2633	error	a computed style must be a style name (a string), found {type}	4.3
P2634	error	the parameter name '{name}' is not an identifier	2.3
P2635	warning	placeholder left in the document: '{text}'	4.2
P2636	error	the heading gets level {level} after level_shift={shift}; the deepest level is 6	10.2, 10.7, П3
P2637	error	'{name}' is not defined in the fragment and cannot be exported	10.4, решение 135
P2638	warning	'@export' in a module has no effect: every name of a module without a leading '_' is public	10.3, решение 135
P2640	error	collection '{name}' differs between the two passes: it depends on its own result (collected() was read before collect() changed it)	4.6, решение 143
P2639	warning	'{path}' lies outside the project: the symbolic link '{link}' leads to '{target}'; keep that directory together with the project to rebuild the document	10.1, решение 136
P2701	warning	the standard library does not support the document language '{lang}'; English forms are used	библ. 1, решение 115
P2702	error	unknown language '{lang}' in lang=; supported: {allowed}	библ. 1, решение 115
P2703	error	{function}: a float is not accepted in money calculations	библ. 3
P2704	error	{function}: the value {value} is out of range ({range})	библ. 4, 5
P2705	error	unknown currency '{code}'	библ. 3
P2706	error	unknown unit '{code}'	библ. 6
P2707	error	units '{a}' and '{b}' measure different quantities	библ. 6
P2708	error	{function}: the case '{value}' is not supported; allowed: {allowed}	библ. 4, 5
P2709	error	{function}: unknown gender '{value}'; allowed: {allowed}	библ. 4
P2710	error	fmt_date: the template shows the time of day {placeholder}	библ. 5
P2711	error	{function}: unknown placeholder '{placeholder}' in the template	библ. 5
P2712	error	{function}: unknown form '{format}'; allowed: {allowed}	библ. 3, 5
P2713	error	calendar: unknown key '{key}'; allowed: {allowed}	библ. 5
P2714	error	calendar: invalid value of '{key}' (weekend: mon … sun; dates without a time of day)	библ. 5

Code	Kind	Message	§
P2715	error	calendar: the date {date} is both a working day and a day off	библ. 5, решение 115
P2716	error	plural: a fractional number needs the form frac=	библ. 7, решение 115
P2717	error	{function}: the step must be greater than zero	библ. 2
P2718	error	split: the weights must be a non-empty list of non-negative numbers with a positive sum	библ. 3
P2719	error	currency '{code}' has no form frac for the language {lang}	библ. 1, 3, решение 115
P2720	error	qty: only a symmetric tolerance is supported (a number in tol=)	библ. 6
P2721	error	{function}: the argument '{arg}' must be {expected}, not {type}	библ. 1–7
P2722	error	{function}: {count} forms are expected for the language {lang}, {given} given	библ. 1, 3, 7
P2723	error	{function}: the fractional part has more than {places} digits	библ. 4
P2724	error	{function}: an item has no field '{key}'	библ. 8
P2725	error	{function}: a value of type {type} cannot be a group key	библ. 8
P2727	error	{function}: the argument {value} is outside the domain of the function	библ. 9
P2728	error	{function}: {size} bytes do not fit a QR code with error correction {ecc} up to version {version}	библ. 10, решение 137
P2729	error	{function}: the character '{char}' cannot be written in {encoding}	библ. 10, решение 137
P3101	error	unknown property '{prop}' of the {kind} style '{name}'	11.3, решение 57
P3102	error	invalid value of '{prop}': expected {expected}	11.3, решение 57
P3103	error	style '{name}' is defined twice in the same layer	11.7
P3104	error	'{name}' is both a paragraph and a character style	11.2
P3105	error	unknown parent style '{parent}' of '{name}'	11.2
P3106	error	the parent '{parent}' of '{name}' is a {kind} style, expected a {expected} style	11.2
P3107	error	cycle of style inheritance: {chain}	11.2
P3108	error	unknown style '{name}'	4.3
P3109	error	'{name}' is a {kind} style; a {expected} style is required here	4.3
P3110	error	invalid style name '{name}': a letter followed by letters, digits, '_', '-'	11.2
P3111	error	the style name '{name}' is already used by a style with other properties	11.4, решение 56
P3112	error	unknown {scale} step '{step}'	11.4
P3113	error	only the parameters before=, after=, indent= are allowed here, not '{name}'	11.4
P3114	error	styles cannot be defined here (the top level of an | import module)	11.7, решение 59
P3115	error	unknown construct '{key}' for structure_styles / |set	4.5, 11.5, решение 60
P3118	warning	direct formatting |fmt: changing the style in Writer will not affect this text	4.4, 11.8, 14.2
P3120	error	unknown structure profile '{name}'; expected business or report	5.1

Code	Kind	Message	§
P3121	error	list style '{name}': the format '{format}' (Cyrillic letters) needs a document in Ukrainian or Russian	11.3, решения 115, 133 (только для стилей, которые документ использует)
P3122	error	list item {number} has no letter: the level allows {count} letters ('{letters}')	11.2, 5.4, П1
P4101	error	invalid ID '{id}': an ASCII letter followed by letters, digits, '_', '-', '.'	5.6
P4102	error	duplicate ID '{id}'	5.6
P4103	error	a fragment with the ID '{id}' is included twice without ns=	5.6, решение 74
P4104	error	unknown argument '{name}' of {command}	5
P4105	error	the argument '{name}' of {command} must be {expected}	5
P4201	error	skipped heading level: level {level} after level {previous}	5.2
P4202	warning	the text of a numbered heading starts with a number; the number is added automatically	5.2
P4203	warning	a subsection follows the clauses of section {number}: clause and subsection numbers may coincide	5.3, решение 68
P4204	error	skipped clause level: a level-{level} clause without a parent clause	5.3
P4205	error	unknown clause '{id}' in continues= (the clause must precede)	5.3
P4206	error	continues='{id}': the target must be a clause of the same section, scope and level	5.3
P4207	error	start= and continues= cannot be used together	5.4
P4208	error	unknown list '{name}' in continues= (the list must precede)	5.4
P4209	error	duplicate list name '{name}'	5.4
P4210	error	duplicate or reserved scope name '{name}'	5.5
P4211	error	annex letter '{letter}' is not among the annex letters '{letters}'	5.5, решение 71
P4212	error	heading level {value} is not an integer from 1 to 6	5.2, решение 114
P4301	error	unknown reference target '{id}'	5.7
P4302	error	the reference target '{id}' is excluded by a condition	5.7, решение 75
P4303	warning	the reference target '{id}' is excluded by the condition {condition} (kept by --review)	5.7, решение 75
P4304	error	unsuitable reference mode '{mode}' for the {kind} '{id}'	5.7, решение 76
P4305	error	no reference labels for the document language '{language}'	5.7, решение 72
P4306	error	no form '{case}' of the label '{key}' for the language '{language}'	5.7, решение 72
P4307	error	unknown reference mode '{mode}'; expected number, label, title or page	5.7
P4308	error	the reference to '{id}' of another document cannot show a page number (mode 'page')	5.7, решение 121
P4309	error	the target '{id}' is found in several referenced documents: {documents}	5.7, решение 121
P4401	error	unknown list kind '{kind}'; expected figure, table, abbr or index	5.9
P4402	error	invalid structure setting {setting}: '{value}'; expected {expected}	5.1–5.10

Code	Kind	Message	§
P5101	error	invalid value of '{name}' in col(): '{value}'	6.3
P5102	error	invalid column width '{value}'; expected "40mm", "30%", "*", "auto" or "20mm..60mm"	6.3
P5103	error	row {row}: the cells take {used} positions, {free} positions are free (spans counted)	6.5, решение 83
P5104	error	rows={rows} goes beyond the end of the table ({remaining} rows remain)	6.5
P5105	error	'{text}' is not a value of the column type {type}	6.8, решение 77
P5106	error	{name}() is a table formula function; it is allowed only in a cell formula |=(…)	6.8, решение 81
P5107	error	unknown table column '{column}'	6.8
P5108	error	row {row} is outside the table (rows 1–{rows})	6.8, решение 81
P5109	error	{function}(): a value of type {type} is not a number	6.8, решение 81
P5110	error	{function}() of no values	6.8, решение 81
P5111	error	cycle of cell formulas: {chain}	6.8
P5112	warning	the metrics of the font '{font}' are not available (font file or fontTools missing); column widths are approximate	6.3
P5113	warning	the minimum widths of the columns {columns} do not fit; the columns are narrowed	6.3
P5114	error	the formula cannot be written as a Writer formula (live=True)	6.8, решение 82
P5115	error	|table(data=…): row {row} has no field '{field}'	6.2, П4
P5301	warning	segment {segment}: there is no block of the track '{track}'; the cell gets a placeholder	7.7, решение 134
P5302	error	'|{command}' {where}	7.7, решение 134
P5303	error	'{what}' is not allowed directly inside '|{container}'	7.7, решение 134
P5304	error	'|{command}' is not allowed inside a segment of |parallel	7.7, решение 134
P5305	error	{command}: {problem}: {value}	7.7, решение 134
P5306	error	repeated segment ID '{id}'	7.7, решение 134
P5307	error	|track("{track}"): {problem}	7.7, решение 134
P5308	warning	the standard library does not support the language '{lang}' of the track '{track}'; English forms are used	7.7, решение 134
P5310	warning	segment {segment}, track '{track}': {found} where the primary track has {expected}	7.7, решение 134
P5311	warning	segment {segment}, track '{track}': numbers and identifiers differ from the primary track: missing {missing}; extra {extra}	7.7, решение 134
P5312	warning	segment {segment}, track '{track}': IDs differ from the primary track: missing {missing}; extra {extra}	7.7, решение 134
P5313	warning	segment {segment}, track '{track}': targets of references differ from the primary track: missing {missing}; extra {extra}	7.7, решение 134
P5314	warning	segment {segment}, track '{track}': names in |(…) differ from the primary track: missing {missing}; extra {extra}	7.7, решение 134
P5315	warning	|parallel: the track '{track}' ends with the {kind} counter {value}, the primary track with {primary}	7.7, решение 134
P5316	warning	the footnotes restart by {restart}: Writer numbers the notes of the tracks of |parallel together with the primary track	7.7, 5.10, решение 134
P5320	warning	segment {segment}, track '{track}': the translation is outdated — the primary block changed after it was confirmed (rev {rev}, now {now})	7.7, решение 134

Code	Kind	Message	§
P5321	warning	segment {segment}, track '{track}': the confirmed block changed after it was confirmed (rev {rev}, now {now})	7.7, решение 134
P6101	error	cannot use the image '{path}': {reason}	8.1, решение 85
P6102	error	invalid value of '{name}' of {command}: '{value}'	7, 8
P6103	error	unknown page style '{name}'	7.2
P6104	error	a page change is not allowed inside {where}	7.3
P6105	error	no content after the page change up to the end of the document or environment	7.3
P6106	error	{command} inside a group kept together	7.4
P6107	warning	rotate= is not output yet: the image is written without rotation	8.5
P6108	warning	the image '{path}' has the EXIF orientation {orientation}; it is not applied — rotate the file itself	8.2, решение 85
P6109	error	invalid watermark '{value}': a text or img(…) is expected	8.6, решение 84
P6110	error	invalid SVG text of svg=: {reason}	8.2, решение 131
P6111	error	the SVG text of svg= must not contain DOCTYPE or ENTITY declarations	8.2, решение 131
P6112	error	the SVG text of svg= refers to '{ref}' outside it: only '#…' and 'data:' references are allowed	8.2, решение 131
P6113	error	|{command}: the image is given by exactly one of a path, an img(…) value, key= or svg=	8.2, 8.3, решение 131
P7001	warning	kind '{kind}' contradicts the sub-extension of '{name}', which implies '{implied}'; the key 'kind' is used	2.2, 14.2, решение 100
P7101	error	a project file must have 'kind: project' in its header, found '{kind}'	10.6, решение 117
P7102	error	invalid value of '{key}' in the project file: expected {expected}	10.6, решение 117
P7103	error	build(): {problem}	10.7, решение 117
P7104	error	the source '{path}' of the build target cannot be used: {reason}	10.7, решение 117
P7105	error	unknown parameter '{name}' of the build target '{source}': it is declared neither in the document nor in the project	10.6, 10.7, решение 117
P7106	error	the parameter '{name}' gets a value of type {type}, but it is declared with type {expected}	10.6, 10.7, решение 117
P7107	error	the build target overrides the key '{key}', which is not in the key map of the project	9.3, 10.7, решение 117
P7108	error	the output '{path}' of the build target is outside the project directory	10.7, решение 117
P7109	error	the output '{path}' is written by several build targets	10.7, решение 117
P7110	error	the document parameter '{name}' has type {type}, but the project parameter of the same name has type {expected}	10.6, решение 117
P7111	error	{function}() is available only in the code section of the project file	10.6, 10.7, решение 117
P7112	error	no key map: key= needs a project file with 'keys'	9.3, решение 117
P7113	error	the key '{key}' points to a part of a file ('{value}'); '#part' is allowed only in |include	9.3, решение 117
P7114	error	key= cannot be combined with a path or part=	9.3, решение 117
P7115	warning	'{name}' is exported by several fragments; on_conflict: {mode} keeps the {mode} export	10.4, решение 117
P7116	error	|document is allowed only on its own line in the text of the root document (also inside |if and |for), not in an environment, a part or a fragment	10.7, решения 117, 118

Code	Kind	Message	§
P7117	warning	the key '{key}' in the header of a part is ignored: styles and structure settings come from the root document and the project	10.7, 14.2, решение 117
P7118	error	invalid or repeated scope '{scope}' of |document	10.7, 5.6, решения 117, 118
P7119	error	the document '{path}' is outside the directory of the project '{project}'	10.6, 14.1, решение 117
P7120	error	{option} cannot be used when building a project: the outputs are given by the targets	14.1, решение 117
P7121	error	--dump-project needs a project file (project.p.prml)	14.1, решение 117
P7122	warning	the project has no build targets	10.7, решение 117
P7123	error	--param '{text}' must be written as NAME=VALUE	14.1, П2
P7124	error	--param: the document '{path}' has no parameter '{name}' (neither its own nor of the project)	14.1, П2
P7125	error	--param cannot be used when building a project: the parameters of targets are given in 'targets' and build()	14.1, 10.6, П2
P7126	error	references: '{name}' is not a document of the project (ref_documents: {known})	10.6, решения 121, 129
P7127	error	the referenced document '{name}' could not be evaluated ({errors} errors); build it on its own to see them	10.6, решение 121
P7128	error	the referenced document '{name}' has neither a source nor a registry ({registry})	10.6, решение 121
P7129	warning	the source of the referenced document '{name}' is not found: reference texts come from its registry {registry}	10.6, решение 121
P7130	error	extends: '{path}' leads to a cycle of project files	10.6, решение 123
P7131	error	extends: the project must lie inside the root of its base project '{path}'	10.6, решение 123
P7132	error	--dump-structure and --dump-styles need a document, not a project file	14.1, решение 124
P8001	warning	'{construct}' is not output yet (stage {stage}); a placeholder marks its place	15
P8002	error	cannot write '{path}': {reason}	14.1
P8003	error	PDF export needs LibreOffice: soffice or libreoffice is not found	14.1
P8004	error	PDF export failed: {reason}	14.1
P8006	warning	this block cannot be written inside a list item yet and is omitted	15
P8007	warning	|fmt: unknown or invalid property '{prop}'	4.4
P8008	warning	page numbers in the PDF (contents, lists, page references) are not updated: {reason}	5.9, 14.1
P8101	warning	'{construct}' has no counterpart in the export and is left out (page layout: headers, footers, page numbers)	14.1, решение 139
P8102	error	the export to this format needs pandoc: pandoc is not found (https://pandoc.org; --to json writes the AST without it)	14.1, решение 139
P8103	error	pandoc failed: {reason}	14.1, решение 139
P8201	error	the import needs pandoc: pandoc is not found (https://pandoc.org)	14.1, решение 146
P8202	error	the import failed: {reason}	14.1, решение 146
P8203	error	unknown format of the source '{suffix}': use --from (docx, odt, rtf, gfm, html, epub, markdown) or a known extension	14.1, решение 146

Code	Kind	Message	§
P8204	error	'{file}' is an old Word file: LibreOffice (soffice) is needed to convert it to DOCX and was not found	14.1, решение 146
P8205	error	'{file}' already exists: it is not replaced (use --force or -o)	14.1, решение 146
P9101	error	unknown admonition kind '{kind}'; allowed: {allowed}	9.1, решение 116
P9102	error	the admonition is empty: no text, hazard= or consequence=	9.1, решение 116
P9103	warning	'{kind}' without hazard=: name the kind and source of the hazard	9.1, 14.2, решение 116
P9104	warning	hazard= and consequence= are ignored for '{kind}'	9.1, решение 116
P9105	warning	the styles module function admonition() is not used in language version 1.1; blocks are formatted by styles	9.1, решение 116
P9106	warning	this |if defines names in its branches and is not shown with all variants in the --review mode	9.2, решение 116
P9201	error	unknown abbreviation '{name}'	9.4, решение 116
P9202	error	the abbreviation '{name}' is defined with different expansions	9.4, решение 116
P9203	error	show=False needs the expansion: |abbr("{name}", "…", show=False)	9.4, решение 116
P9204	warning	abbreviations(first="short") without |list_of("abbr"): the expansions are written nowhere	9.4, решение 116
P9205	warning	|list_of("abbr"): the document defines no abbreviations; the list is not written	5.9, 9.4, решение 116
P9206	error	abbreviations({key}=…): invalid value '{value}'; allowed: {allowed}	9.4, решение 116
P9207	error	|abbr needs the abbreviation as the first argument (a string)	9.4, решение 116
P9301	error	|index needs a non-empty term (a string) and an optional sub= string	9.5, решение 116

Appendix E. Styles of the kit

Generated by `tools/author_docs.py appendices` from the reference of the translator; do not edit.

std/styles/default

The built-in base: styles of the constructs, used when no styles module is named.

Base: none. Default font: Liberation Sans, 11pt.

Style	Type	Properties in the module
`AbbreviationList`	table	frame: none, rules: none, padding: [0.5mm, 2mm, 0.5mm, 0mm], repeat_header: false, parent: TableGrid
`Admonition`	paragraph	keep_together: true, parent: Body
`AdmonitionCaution`	paragraph	bold: true, color: #9C6500, parent: AdmonitionText
`AdmonitionDanger`	paragraph	bold: true, color: #C00000, parent: AdmonitionText
`AdmonitionNotice`	paragraph	bold: true, color: #1F4E9E, parent: AdmonitionText
`AdmonitionTable`	table	frame: none, rules: none, padding: [1mm, 2mm, 1mm, 2mm], paragraph: AdmonitionText, may_break: false, space_before: 3mm, space_after: 3mm, parent: TableGrid
`AdmonitionTableCaution`	table	parent: AdmonitionTable
`AdmonitionTableDanger`	table	parent: AdmonitionTable
`AdmonitionTableNotice`	table	parent: AdmonitionTable
`AdmonitionTableWarning`	table	parent: AdmonitionTable
`AdmonitionText`	paragraph	align: left, space_after: 1mm, keep_with_next: false, parent: Body
`AdmonitionWarning`	paragraph	bold: true, color: #E36C09, parent: AdmonitionText
`AppendixDesignation`	paragraph	align: right, keep_with_next: true, space_after: 0mm, parent: Body
`AppendixSubtitle`	paragraph	align: right, keep_with_next: true, space_after: 2mm, parent: Body
`AppendixTitle`	paragraph	bold: true, align: center, keep_with_next: true, space_before: 2mm, space_after: 4mm, parent: Body
`Body`	paragraph	align: justify, line_height: 115%, space_after: 2mm, first_line: 0mm, widows: 2, orphans: 2
`BodyCentered`	paragraph	align: center, parent: Body
`BodyCompact`	paragraph	size: 10pt, line_height: 110%, space_after: 1mm, parent: Body
`BodyIndented`	paragraph	first_line: 10mm, parent: Body
`BodyLeft`	paragraph	align: left, parent: Body
`BodyRight`	paragraph	align: right, parent: Body
`BulletList`	list	levels: {1: {type: bullet, bullet: •, left: 5mm, hanging: 5mm}, 2: {type: bullet, bullet: –, left: 10mm, hanging: 5mm}, 3: {type: bullet, bullet: ◦, left: 15mm, hanging: 5mm}}
`Checklist`	list	paragraph: ListText, levels: {1: {type: bullet, bullet: ☐, left: 8mm, hanging: 5mm}}
`Clause`	paragraph	parent: Body
`Code`	character	font: Liberation Mono
`Contents1`	paragraph	align: left, space_after: 1mm, parent: Body
`Contents2`	paragraph	left: 5mm, parent: Contents1
`Contents3`	paragraph	left: 10mm, parent: Contents1
`Contents4`	paragraph	left: 15mm, parent: Contents1

Style	Type	Properties in the module
`Contents5`	paragraph	left: 20mm, parent: Contents1
`Contents6`	paragraph	left: 25mm, parent: Contents1
`ContentsTitle`	paragraph	bold: true, size: 12pt, align: center, keep_with_next: true, space_after: 4mm, parent: Body
`DashList`	list	paragraph: ListText, levels: {1: {type: bullet, bullet: –, left: 7mm, hanging: 4mm}, 2: {type: bullet, bullet: –, left: 14mm, hanging: 4mm}}
`DefinedTerm`	character	bold: true
`Emphasis`	character	italic: true
`Endnote`	paragraph	parent: Footnote
`Figure`	paragraph	align: center, keep_with_next: true, space_before: 2mm, space_after: 1mm, parent: Body
`FigureCaption`	paragraph	align: center, space_before: 1mm, space_after: 2mm, parent: Body
`Footer`	paragraph	size: 9pt, align: center, space_after: 0mm, parent: Body
`Footnote`	paragraph	size: 9pt, align: left, line_height: 100%, space_after: 0mm, left: 4mm, first_line: -4mm, parent: Body
`Group`	block	keep_together: true
`Header`	paragraph	size: 9pt, align: center, space_after: 0mm, parent: Body
`Heading`	paragraph	bold: true, align: left, keep_with_next: true, space_before: 4mm, space_after: 2mm, parent: Body
`Heading1`	paragraph	size: 14pt, parent: Heading
`Heading2`	paragraph	size: 13pt, parent: Heading
`Heading3`	paragraph	size: 12pt, parent: Heading
`Heading4`	paragraph	size: 11pt, parent: Heading
`Heading5`	paragraph	size: 11pt, italic: true, parent: Heading
`Heading6`	paragraph	size: 11pt, bold: false, italic: true, parent: Heading
`IndexEntry`	paragraph	align: left, space_after: 1mm, parent: Body
`IndexEntry2`	paragraph	left: 5mm, parent: IndexEntry
`Landscape`	page	orientation: landscape, parent: Portrait
`LatinLetterList`	list	paragraph: ListText, levels: {1: {type: number, format: a, suffix:), left: 9mm, hanging: 6mm}, 2: {type: bullet, bullet: –, left: 18mm, hanging: 4mm}}
`LetterList`	list	paragraph: ListText, levels: {1: {type: number, format: a, suffix:), left: 9mm, hanging: 6mm}, 2: {type: bullet, bullet: –, left: 18mm, hanging: 4mm}}
`ListText`	paragraph	align: left, space_after: 1mm, parent: Body
`MixedList`	list	paragraph: ListText, levels: {1: {type: number, format: 1, suffix: ., left: 9mm, hanging: 6mm}, 2: {type: bullet, bullet: –, left: 18mm, hanging: 4mm}}
`Note`	paragraph	size: 9pt, italic: true, align: left, parent: Body
`NoteAnchor`	character	position: super
`NoteLabel`	character	
`NoteList`	list	paragraph: Note, levels: {1: {type: number, format: 1, suffix: ., left: 5mm, hanging: 5mm}}
`NumberedList`	list	levels: {1: {type: number, format: 1, suffix: ., left: 6mm, hanging: 6mm}, 2: {type: number, format: 1, suffix: ., left: 12mm, hanging: 8mm}, 3: {type: number, format: a, suffix:), left: 18mm, hanging: 6mm}}
`ParallelHeader`	paragraph	bold: true, align: left, keep_with_next: true, parent: Body
`ParallelTable`	table	width: [percent, 100], frame: none, rules: none, padding: 0mm, repeat_header: true, space_before: 3mm, space_after: 3mm
`Placeholder`	character	background: #FFFF00
`Portrait`	page	size: A4, orientation: portrait, margin: [20mm, 15mm, 20mm, 25mm]

Style	Type	Properties in the module
`Preformatted`	paragraph	font: Liberation Mono, size: 9pt, align: left, line_height: 100%, space_after: 0mm, widows: 1, orphans: 1, parent: Body
`Quote`	paragraph	italic: true, left: 7mm, right: 5mm, space_before: 2mm, space_after: 3mm, parent: Body
`Reference`	character	
`ReviewFalse`	character	background: #F8D7D7
`ReviewLabel`	paragraph	size: 8pt, italic: true, color: #555555, align: left, space_after: 0mm, keep_with_next: true, parent: Body
`ReviewTrue`	character	background: #D9F2D9
`SmallPrint`	paragraph	align: left, size: 9pt, line_height: 110%, space_after: 1.5mm, color: #333333, parent: Body
`SmallText`	character	size: 9pt
`StrikeText`	character	strike: true
`Strong`	character	bold: true
`SubClause`	paragraph	left: 5mm, parent: Clause
`Subscript`	character	position: sub
`Superscript`	character	position: super
`TableCaption`	paragraph	align: left, keep_with_next: true, space_before: 2mm, space_after: 1mm, parent: Body
`TableCompact`	table	padding: 0.8mm, paragraph: TableTextCompact, header_paragraph: TableHeaderCompact, parent: TableGrid
`TableGrid`	table	width: [percent, 100], frame: 0.5pt solid #000000, rules: all, padding: 1mm, paragraph: TableText, header_bold: true, repeat_header: true
`TableHeader`	paragraph	bold: true, parent: TableText
`TableHeaderCompact`	paragraph	bold: true, parent: TableTextCompact
`TableNoBorders`	table	frame: none, rules: none, padding: [1mm, 2mm, 1mm, 0mm], repeat_header: false, header_background: none, parent: TableGrid
`TableText`	paragraph	align: left, space_after: 0mm, parent: Body
`TableTextCompact`	paragraph	size: 9pt, line_height: 105%, parent: TableText
`Underline`	character	underline: true

std/styles/business

Contracts, acts, invoices, protocols; margins and indents after DSTU 4163:2020.

Base: `std/styles/default`. Default font: Liberation Serif, 12pt.

Style	Type	Properties in the module
`Approval`	paragraph	left: 100mm, space_after: 0mm, keep_with_next: true, keep_together: true, parent: BodyLeft
`ApprovalTitle`	paragraph	bold: true, space_after: 1mm, parent: Approval
`AttachmentLabel`	paragraph	size: 10pt, space_before: 3mm, keep_with_next: true, parent: BodyLeft
`Body` (adds to)	paragraph	align: justify, line_height: 115%, space_after: 2mm
`ClauseContinuation`	paragraph	left: 7mm, parent: Body
`DatePlace`	paragraph	space_before: 2mm, space_after: 4mm, keep_with_next: true, parent: BodyLeft
`Details`	paragraph	size: 10pt, line_height: 110%, space_after: 1mm, parent: BodyLeft
`DetailsTitle`	paragraph	bold: true, size: 11pt, space_after: 2mm, keep_with_next: true, parent: Details
`DocumentSubtitle`	paragraph	space_after: 2mm, keep_with_next: true, keep_together: true, parent: BodyCentered

Style	Type	Properties in the module
`DocumentTitle`	paragraph	size: 14pt, bold: true, space_before: 3mm, space_after: 2mm, keep_with_next: true, keep_together: true, parent: BodyCentered
`DocumentVersion`	paragraph	size: 9pt, color: #444444, space_after: 4mm, keep_with_next: true, parent: BodyCentered
`FigureCaption` (adds to)	paragraph	space_after: 3mm, size: 10pt
`Footer` (adds to)	paragraph	color: #333333
`FormLabel`	paragraph	bold: true, space_after: 1mm, keep_with_next: true, parent: BodyLeft
`Header` (adds to)	paragraph	color: #333333
`Heading` (adds to)	paragraph	align: left, keep_together: true
`Heading1` (adds to)	paragraph	size: 12pt, space_before: 4mm, space_after: 2mm
`Heading2` (adds to)	paragraph	size: 12pt, space_before: 3mm, space_after: 1.5mm
`Heading3` (adds to)	paragraph	size: 12pt, space_before: 2mm, space_after: 1mm
`Heading4` (adds to)	paragraph	size: 12pt, italic: true, space_before: 2mm, space_after: 1mm
`Illustration`	frame	width: 110mm
`IllustrationSmall`	frame	width: 70mm
`IllustrationWide`	frame	width: 160mm
`Logo`	frame	width: 40mm
`LogoSmall`	frame	width: 25mm
`Note` (adds to)	paragraph	size: 10pt, italic: false, space_before: 1mm
`OrganizationName`	paragraph	bold: true, space_after: 2mm, keep_with_next: true, parent: BodyLeft
`Placeholder` (adds to)	character	background: #FFF2CC
`Portrait` (adds to)	page	margin: [20mm, 10mm, 20mm, 30mm], footer_gap: 3mm, footer_min_height: 5mm
`Preamble`	paragraph	space_after: 3mm, parent: Body
`SignatureLine`	paragraph	space_after: 3mm, keep_together: true, parent: BodyLeft
`SignatureRole`	paragraph	space_before: 5mm, space_after: 2mm, keep_with_next: true, keep_together: true, parent: BodyLeft
`TableCaption` (adds to)	paragraph	space_before: 3mm, space_after: 1.5mm, size: 10pt
`TableForm`	table	cols: [30%, 70%], parent: TableGrid
`TableGrid` (adds to)	table	frame: 0.5pt solid #999999, padding: 1.5mm, border: 0.5pt solid #999999, header_background: #EEEEEE, header_paragraph: TableHeader, keep_rows: true, space_before: 1mm, space_after: 3mm
`TableParties`	table	cols: [50%, 50%], paragraph: Details, header_paragraph: DetailsTitle, header_background: none, padding: [0.5mm, 3mm, 0.5mm, 0mm], may_break: false, parent: TableNoBorders
`TableSchedule`	table	cols: [35%, 45%, 20%], parent: TableGrid
`TableServices`	table	cols: [52%, 12%, 18%, 18%], parent: TableGrid
`TableText` (adds to)	paragraph	size: 10pt, line_height: 110%, widows: 1, orphans: 1

std/styles/letter

Official letters after DSTU 4163:2020.

Base: `std/styles/business`. Default font: Liberation Serif, 12pt.

Style	Type	Properties in the module
`BodyIndented` (adds to)	paragraph	space_after: 1mm
`Executor`	paragraph	space_before: 8mm, space_after: 0mm, keep_together: true, parent: SmallPrint
`LetterFirst`	page	next: Portrait, parent: Portrait

Style	Type	Properties in the module
`LetterSubject`	paragraph	bold: true, space_before: 6mm, space_after: 4mm, right: 97mm, keep_with_next: true, parent: BodyLeft
`LetterheadDetails`	paragraph	size: 9pt, line_height: 105%, space_after: 0mm, right: 97mm, parent: BodyLeft
`LetterheadName`	paragraph	bold: true, size: 12pt, space_after: 1mm, right: 97mm, parent: BodyLeft
`OutgoingNumber`	paragraph	size: 11pt, space_before: 4mm, space_after: 0mm, right: 97mm, parent: BodyLeft
`Recipient`	paragraph	left: 90mm, line_height: 100%, space_after: 1.5mm, keep_with_next: true, parent: BodyLeft
`Salutation`	paragraph	space_after: 3mm, keep_with_next: true, parent: BodyCentered

std/styles/technical

Manuals, instructions, technical descriptions.

Base: `std/styles/default`. Default font: Liberation Sans, 10.5pt.

Style	Type	Properties in the module
`Body` (adds to)	paragraph	align: left, line_height: 120%, space_after: 2mm
`Code` (adds to)	character	background: #F2F2F2
`Command`	character	font: Liberation Mono, bold: true
`DefinitionTerm`	paragraph	bold: true, space_after: 0.5mm, keep_with_next: true, parent: Body
`DefinitionText`	paragraph	left: 8mm, space_after: 2mm, parent: Body
`FileName`	character	font: Liberation Mono
`Heading1` (adds to)	paragraph	size: 15pt, space_before: 6mm, space_after: 3mm
`Heading2` (adds to)	paragraph	size: 13pt, space_before: 5mm, space_after: 2mm
`Heading3` (adds to)	paragraph	size: 11.5pt, space_before: 4mm, space_after: 1.5mm
`Key`	character	font: Liberation Mono, bold: true, background: #E8E8E8
`MenuItem`	character	bold: true
`Portrait` (adds to)	page	margin: [20mm, 15mm, 20mm, 25mm]
`Preformatted` (adds to)	paragraph	size: 9pt, space_after: 2mm, left: 3mm, right: 3mm, background: #F2F2F2, space_before: 1mm
`ProgramOutput`	paragraph	color: #333333, parent: Preformatted
`StepList`	list	paragraph: ListText, levels: {1: {type: number, format: 1, suffix: ., left: 8mm, hanging: 8mm}, 2: {type: number, format: a, suffix:), left: 16mm, hanging: 6mm}}
`StepResult`	paragraph	italic: true, left: 8mm, space_after: 2mm, parent: Body
`TableGrid` (adds to)	table	frame: 0.5pt solid #808080, border: 0.5pt solid #808080, header_background: #E6E6E6, header_paragraph: TableHeader
`TableRevisions`	table	cols: [16mm, 24mm, *, 40mm], parent: TableCompact
`Variable`	character	font: Liberation Mono, italic: true

std/styles/report

Reports after DSTU 3008:2015.

Base: `std/styles/default`. Default font: Liberation Serif, 14pt.

Style	Type	Properties in the module
`AppendixDesignation` (adds to)	paragraph	align: center, first_line: 0mm, bold: true
`AppendixSubtitle` (adds to)	paragraph	align: center, first_line: 0mm
`AppendixTitle` (adds to)	paragraph	bold: false, first_line: 0mm

Style	Type	Properties in the module
`Body` (adds to)	paragraph	align: justify, line_height: 150%, space_after: 0mm, first_line: 12.5mm
`Contents1` (adds to)	paragraph	space_after: 0mm, first_line: 0mm
`ContentsTitle` (adds to)	paragraph	parent: StructuralHeading
`Figure` (adds to)	paragraph	first_line: 0mm
`FigureCaption` (adds to)	paragraph	space_before: 1mm, space_after: 3mm, first_line: 0mm
`Footer` (adds to)	paragraph	first_line: 0mm
`Footnote` (adds to)	paragraph	size: 12pt
`Header` (adds to)	paragraph	size: 12pt, align: right, first_line: 0mm
`Heading` (adds to)	paragraph	align: left, space_before: 7mm, space_after: 7mm, first_line: 12.5mm, keep_together: true
`Heading1` (adds to)	paragraph	size: 14pt, odf: {fo:text-transform: uppercase}
`Heading2` (adds to)	paragraph	size: 14pt
`Heading3` (adds to)	paragraph	size: 14pt
`Heading4` (adds to)	paragraph	size: 14pt, italic: false
`ListText` (adds to)	paragraph	space_after: 0mm, first_line: 0mm, line_height: 150%
`Note` (adds to)	paragraph	size: 12pt, italic: false, align: justify, line_height: 100%, first_line: 12.5mm
`ParallelHeader` (adds to)	paragraph	first_line: 0mm
`Portrait` (adds to)	page	margin: [20mm, 10mm, 20mm, 25mm]
`Preformatted` (adds to)	paragraph	size: 12pt, first_line: 0mm
`ReportTitle`	page	next: Portrait, parent: Portrait
`StructuralHeading`	paragraph	align: center, first_line: 0mm, page_break_before: true, parent: Heading1
`TableCaption` (adds to)	paragraph	space_before: 3mm, space_after: 1mm, first_line: 12.5mm
`TableText` (adds to)	paragraph	first_line: 0mm, size: 12pt, line_height: 100%
`TitlePageName`	paragraph	bold: true, space_before: 40mm, space_after: 6mm, odf: {fo:text-transform: uppercase}, parent: TitlePageText
`TitlePageText`	paragraph	first_line: 0mm, space_after: 2mm, parent: BodyCentered

std/styles/compact

Memos, quick references, cheat sheets.

Base: `std/styles/default`. Default font: Liberation Sans, 9pt.

Style	Type	Properties in the module
`Body` (adds to)	paragraph	align: left, line_height: 112%, space_after: 1mm, widows: 1, orphans: 1
`BulletList` (adds to)	list	levels: {1: {type: bullet, bullet: •, left: 4mm, hanging: 3mm}, 2: {type: bullet, bullet: –, left: 8mm, hanging: 3mm}, 3: {type: bullet, bullet: ◦, left: 12mm, hanging: 3mm}}
`Footer` (adds to)	paragraph	size: 7.5pt
`Header` (adds to)	paragraph	size: 7.5pt
`Heading` (adds to)	paragraph	space_before: 2.5mm, space_after: 1mm
`Heading1` (adds to)	paragraph	size: 12pt
`Heading2` (adds to)	paragraph	size: 10.5pt
`Heading3` (adds to)	paragraph	size: 9.5pt
`Heading4` (adds to)	paragraph	size: 9pt
`Heading5` (adds to)	paragraph	size: 9pt

Style	Type	Properties in the module
`Heading6` (adds to)	paragraph	size: 9pt
`ListText` (adds to)	paragraph	space_after: 0.5mm
`Note` (adds to)	paragraph	size: 8pt
`Portrait` (adds to)	page	margin: [12mm, 12mm, 12mm, 12mm]
`Preformatted` (adds to)	paragraph	size: 8pt, background: #F2F2F2, left: 1mm, right: 1mm
`SmallPrint` (adds to)	paragraph	size: 7.5pt
`TableGrid` (adds to)	table	frame: 0.4pt solid #808080, padding: 0.7mm, border: 0.4pt solid #808080, header_background: #E6E6E6, header_paragraph: TableHeader
`TableText` (adds to)	paragraph	size: 8.5pt, line_height: 105%
`TableTextCompact` (adds to)	paragraph	size: 8pt

Глоссарий терминов Primula (ru / uk / en)

Единые термины руководства автора, справки Studio, сообщений транслятора и переводов. Переводчик (человек или модель по зафиксированному промпту этапа 15.5) берёт термин отсюда, а не подбирает синоним. Новый термин сначала вносится сюда. Записи `|…` — конструкции языка, они не переводятся.

ru	uk	en	Пояснение
документ	документ	document	файл `*.prml`, собирается в ODT
фрагмент	фрагмент	fragment	файл `*.f.prml`, подключается `|include`
модуль	модуль	module	файл `*.m.prml`, только определения, `|import`
модуль стилей	модуль стилів	styles module	файл `*.s.prml`
проект	проєкт	project	файл `project.p.prml`: параметры, ключи, цели
часть (сводного документа)	частина	part	`|document` или `|part`
заголовок YAML	заголовок YAML	YAML header	блок между строками `---` в начале файла
ключ заголовка	ключ заголовка	header key	`doc:`, `params:`, `styles:` …
метаданные	метадані	metadata	ключ `doc:`; в проекте `doc_defaults:`
параметр	параметр	parameter	`params:`, `--param`
карта ключей	карта ключів	key map	`keys:` проекта для `key=`
цель (сборки)	ціль	target	`targets:` проекта, `build()`
сборка	збирання	build	`primula build`
проверка	перевірка	check	`primula check`
транслятор	транслятор	translator	программа `primula`
разметка	розмітка	markup	текст с командами `|`
команда	команда	command	`|имя(…)`
длинная форма	довга форма	long form	`|имя(…)`, `|имя(…):`
краткая форма	коротка форма	short form	`|>`, `|*{…}` …
строчная форма	рядкова форма	line form	команда до конца строки
окружение	оточення	environment	`|имя(…):` … `|end имя`
маркер закрытия	маркер закриття	closing marker	`|end`, `|endif` …
аргумент	аргумент	argument	значение в `(…)` команды
тело	тіло	body	`{…}` команды или окружение
выражение	вираз	expression	`|(…)`, аргументы команд
блок кода	блок коду	code block	`|code:` … `|end`
вычислительное ядро	обчислювальне ядро	core (evaluator)	подмножество Python
определение	визначення	definition	имя, заданное кодом, `|let`, `|def`
макрос	макрос	macro	`|def` … `|enddef`
действие	дія	action	оператор верхнего уровня кода, кроме определения: `for`, `if`, вызов
запись	запис	record	значение `record`: поля через точку
экспорт	експорт	export	передача имени фрагмента документу: `@export`, `|export`, `export=True`
декоратор	декоратор	decorator	`@export` над `def`

ru	uk	en	Пояснение
стандартная библиотека	стандартна бібліотека	standard library	функции `num`, `money`, `fmt_date` …
библиотека	бібліотека	library	каталог модулей и данных для нескольких проектов (11)
символическая ссылка	символічне посилання	symbolic link	`lib` → `../shared/`…: подключение библиотеки (решение 136)
документ-тест	документ-тест	test document	документ, который проверяет библиотеку через `fail()`
заголовок (раздела)	заголовок	heading	`|<` … `|<<<<<<`
пункт	пункт	clause	`|>`, нумеруется по уровню
элемент списка	елемент списку	list item	`|-`, `|#`
предупреждающий блок	застережний блок	admonition	`|!("warning")`
приложение (к документу)	додаток	appendix	`|appendix`
перекрёстная ссылка	перехресне посилання	cross reference	`|@("id")`
идентификатор	ідентифікатор	ID	`id=` цели ссылки
сноска	виноска	footnote	`|footnote`
оглавление	зміст	table of contents	`|toc`
таблица	таблиця	table	`|table`
строка (таблицы)	рядок	row	`|:`
ячейка	клітинка	cell	`|&`
столбец	стовпець	column	`cols=`
шапка таблицы	шапка таблиці	header rows	`header=`
итоговые строки	підсумкові рядки	footer rows	`footer=`
формула ячейки	формула клітинки	cell formula	`|=(…)`
стиль	стиль	style	имя в модуле стилей
абзацный стиль	стиль абзацу	paragraph style	`|.Имя:`
символьный стиль	символьний стиль	character style	`|.Имя{…}`
стиль страницы	стиль сторінки	page style	`page_style`, `|page`
слой стилей	шар стилів	style layer	проект → `--style` → документ
набор стилей (комплекта)	набір стилів	styles set	`std/styles/business` … (раздел 11.9)
основа модуля стилей	основа модуля стилів	base (of a styles module)	`extends:` модуля стилей
раздел с колонками	розділ з колонками	section	`|section(cols=…)`
параллельные тексты	паралельні тексти	parallel texts	`|parallel`
поток (параллельного текста)	потік	track	`track(…)`, `|track`
основной поток	основний потік	primary track	`primary=`
сегмент	сегмент	segment	`|segment`
сегмент на всю ширину	сегмент на всю ширину	full-width segment	`span="all"`
проверка соответствия	перевірка відповідності	correspondence check	`parity=`
подтверждение перевода	підтвердження перекладу	confirmation of a translation	`primula parallel --stamp`, `rev=`
перевод устарел	переклад застарів	the translation is outdated	P5320
колонтитул	колонтитул	header / footer	`|header`, `|footer`

ru	uk	en	Пояснение
рисунок	рисунок	figure	`|figure`
изображение	зображення	image	`|img`, `img(…)`
водяной знак	водяний знак	watermark	`|watermark`
заполнитель	заповнювач	placeholder	`|?{…}`
буквальный текст	буквальний текст	verbatim text	`|verbatim`
комментарий	коментар	comment	`|%`, `|comment`
сообщение (диагностики)	повідомлення	diagnostic	`error[P1209]`
ошибка / предупреждение	помилка / попередження	error / warning	серьёзность сообщения
консоль вычислителя	консоль обчислювача	evaluator console	`primula console`